

PreOne Enterprise

OpenAPI Specification

OpenAPI 3.1 — Enterprise Preschool Operating System

Document Version: 1.0

Status: API Specification Freeze

Specification: OpenAPI 3.1

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: Vision v1.0, Master PRD v1.0, DDD v1.0, ERD v3.0, API Contract Catalog v1.0

Successor: Swagger UI, Postman Collection, SDK Generation, Mock Server, Backend Validation, API Testing

Scope: 14 domains, ~530 REST APIs, 5 WebSocket channels, 5 webhook events

Classification: Internal Engineering Reference

Prepared by: PreOne Architecture & Engineering Team

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	2
1.3 Audience.....	3
1.4 Specification Status.....	5
1.5 Related Documents.....	5
2. OpenAPI Standards.....	5
2.1 Specification Version.....	5
2.2 Standards Compliance.....	6
2.3 File Organization.....	6
2.4 Reference Strategy.....	10
3. API Information.....	11
3.1 info Object.....	11
3.2 Field Reference.....	14
3.3 Versioning Strategy.....	14
4. Servers.....	15
4.1 Server Environments.....	15
4.2 Environment Reference.....	17
4.3 Environment Selection.....	17
5. Security Schemes.....	18
5.1 Supported Schemes.....	18
5.2 Scheme Reference.....	22
5.3 Future-Ready Authentication.....	22
6. Authentication APIs.....	23

6.1 Login Flow.....	23
6.2 Login Endpoint.....	23
6.3 Authentication Endpoints.....	28
6.4 Token Lifecycle.....	29
7. Authorization.....	30
7.1 Scope-Based Access Control.....	30
7.2 Scope Catalog.....	30
7.3 Authorization Flow.....	30
8. Common Headers.....	34
8.1 Multi-Tenant Header Standards.....	34
8.2 Header Reference.....	34
8.3 Header Examples.....	34
9. Request Schemas.....	38
9.1 Schema Standards.....	38
9.2 Request Patterns.....	38
9.3 StudentCreateRequest Schema.....	39
10. Response Schemas.....	46
10.1 Response Standards.....	46
10.2 Response Patterns.....	46
10.3 StudentResponse Schema.....	46
10.4 Pagination Response.....	52
11. Error Schemas.....	57
11.1 Standard Error Envelope.....	57
11.2 Common Status Codes.....	64
11.3 Error Code Catalog.....	64
12. Common Components.....	64
12.1 Components Overview.....	64
12.2 Component Reference.....	65

12.3 Reusable Parameter Example.....	65
12.4 Reusable Response Example.....	71
13. Identity APIs.....	75
13.1 Identity Domain Overview.....	75
13.2 Identity Endpoints.....	75
13.3 Create User Example.....	75
14. CRM APIs.....	80
14.1 CRM Domain Overview.....	81
14.2 CRM Endpoints.....	81
14.3 Lead Conversion Example.....	81
15. Admission APIs.....	85
15.1 Admission Domain Overview.....	85
15.2 Admission Endpoints.....	85
15.3 Approve Application Example.....	85
16. Student APIs.....	88
16.1 Student Domain Overview.....	88
16.2 CRUD Pattern.....	89
16.3 Student Endpoints.....	90
16.4 Student Example.....	91
17. Academics APIs.....	94
17.1 Academics Domain Overview.....	94
17.2 Academics Endpoints.....	94
17.3 Bulk Marks Entry Example.....	95
18. Attendance APIs.....	100
18.1 Attendance Domain Overview.....	100
18.2 Attendance Endpoints.....	100
18.3 Bulk Attendance Example.....	100
19. Communication APIs.....	105

19.1 Communication Domain Overview.....	106
19.2 Communication Endpoints.....	106
19.3 Send Message Example.....	106
20. Finance APIs.....	109
20.1 Finance Domain Overview.....	110
20.2 Finance Endpoints.....	110
20.3 Create Invoice Example.....	110
21. Inventory APIs.....	114
21.1 Inventory Domain Overview.....	114
21.2 Inventory Endpoints.....	115
21.3 Stock Movement Example.....	115
22. HR APIs.....	118
22.1 HR Domain Overview.....	118
22.2 HR Endpoints.....	118
22.3 Apply Leave Example.....	118
23. Administration APIs.....	121
23.1 Administration Domain Overview.....	121
23.2 Administration Endpoints.....	121
23.3 Audit Log Query Example.....	122
24. Reports APIs.....	126
24.1 Reports Domain Overview.....	126
24.2 Reports Endpoints.....	126
24.3 Generate Report Example.....	127
25. Platform APIs.....	131
25.1 Platform Domain Overview.....	131
25.2 Platform Endpoints.....	131
25.3 Register Webhook Example.....	131
26. WebSocket Documentation.....	136

26.1 WebSocket Overview.....	136
26.2 Documented Channels.....	136
26.3 Connection Example.....	136
27. Webhooks.....	141
27.1 Webhook Overview.....	141
27.2 Webhook Events.....	142
27.3 Webhook Payload Example.....	142
28. Examples.....	147
28.1 Per-Endpoint Example Matrix.....	147
28.2 Pagination Example.....	147
28.3 Filter Example.....	150
28.4 File Upload Example.....	151
29. Tags.....	154
29.1 Tag Purpose.....	154
29.2 Tag Catalog.....	154
29.3 Tag Definition Example.....	154
30. Versioning.....	158
30.1 Versioning Strategy.....	158
30.2 Versioning Rules.....	159
30.3 Breaking Change Detection.....	159
31. Deprecation Policy.....	161
31.1 Deprecation Lifecycle.....	161
31.2 Deprecation Stages.....	161
31.3 Deprecation Spec Example.....	161
32. SDK Generation.....	165
32.1 SDK Targets.....	166
32.2 SDK Targets Catalog.....	166
32.3 TypeScript SDK Example.....	166

33. Validation Rules.....	170
33.1 Validation Standards.....	170
33.2 Validation Rules Catalog.....	171
33.3 Validation Example.....	171
34. Governance.....	177
34.1 Governance Overview.....	178
34.2 Governance Roles.....	178
34.3 Governance Workflow.....	178
34.4 Governance Tools.....	178
35. Generated Deliverables.....	185
35.1 Automation Pipeline.....	185
35.2 Deliverables Catalog.....	185
35.3 Quality Gates.....	186
36. Glossary.....	187
36.1 Terms.....	187
37. Document Control.....	187
37.1 Document Information.....	187
37.2 Change History.....	187
37.3 Approval Matrix.....	188
37.4 Next Steps.....	188

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Introduction

1.1 Purpose

हा Document PreOne Enterprise API ची complete OpenAPI 3.1 specification प्रदान करतो. PreOne हे Enterprise Preschool Operating System आहे जे 14 business domains, 530+ REST endpoints, 5 WebSocket channels, आणि 5 webhook events expose करते. या document चा primary goal म्हणजे एक canonical, machine-readable API contract स्थापित करणे ज्यावरून Swagger UI, Postman collections, mock servers, आणि multi-language SDKs auto-generate होतील.

OpenAPI Specification (OAS) ही industry-standard format आहे HTTP APIs describe करण्यासाठी. ही specification फक्त documentation नाही — ती एक living contract आहे जी backend implementation, frontend consumption, mobile integration, आणि third-party partner integrations सर्वाना bind करते. CI pipeline मध्ये ही spec validate होते आणि प्रत्येक PR मध्ये breaking change detection असते.

या specification वरून पुढील artifacts automatically generate होतील: Swagger UI (interactive API explorer), OpenAPI YAML/JSON files, Postman Collection, Mock Server (Prism-based), TypeScript SDK, Flutter/Dart SDK, C# SDK, Java SDK, API Test Collections (Postman + Newman), Contract Validation suite, आणि client code generation 10 target languages साठी.

1.2 Scope

ही specification PreOne platform च्या सर्व public आणि internal APIs cover करते. खालील explicitly in scope आहेत:

- Authentication APIs (login, OTP, refresh, logout)
- 14 business domain APIs (Identity, CRM, Admissions, Students, Academics, Attendance, Communication, Finance, Inventory, HR, Administration, Reports, Settings, Platform)
- WebSocket documentation for 5 real-time channels
- Webhook documentation for 5 outbound event types
- Common components (schemas, parameters, headers, responses, examples)
- Security schemes, scopes, common headers, error envelope

खालील explicitly out of scope आहेत:

- Internal microservice-to-microservice contracts (separate internal spec)

- Database schema (covered in ERD v3.0 + Prisma Schema v3.0)
- Backend implementation patterns (covered in Backend TD v1.0)
- Frontend consumption patterns (covered in Frontend Architecture v1.0)
- Infrastructure deployment (covered in DevOps runbook)

1.3 Audience

ही specification खालील audiences साठी लिखित आहे:

- **Backend Engineers:** Implementation reference — controllers, DTOs, validation, error responses mirror the spec exactly.
- **Frontend Engineers:** API consumption patterns — types generated from spec via openapi-typescript.
- **Mobile Engineers:** SDK consumption — Dart SDK auto-generated from this spec.
- **Integration Partners:** Third-party integration — Postman collection + SDKs derived from this spec.
- **QA Engineers:** Contract testing — Postman + Newman tests generated from spec.
- **Product Managers:** API discovery — Swagger UI for exploring available endpoints.
- **AI Code Assistants:** Structured prompt input — spec provides complete contract for code generation.

1.4 Specification Status

ही specification 'API Specification Freeze' status मध्ये आहे. Freeze म्हणजे v1.0 release साठी contract lock झाला आहे — कोणतेही breaking changes नाही, फक्त backward-compatible additions (MINOR bumps) आणि bug fixes (PATCH bumps) allowed आहेत. Breaking changes require MAJOR bump (v2.0) आणि 12-month deprecation cycle.

Specification freeze नंतर कोणताही change फक्त PR-based approval workflow through जाऊन होऊ शकतो. PR मध्ये automated linting (Redocly + Spectral), breaking change detection (oasdiff), आणि minimum 2 reviewers ची sign-off आवश्यक आहे. Breaking changes साठी API Owner + API Architect दोघांची approval mandatory आहे.

1.5 Related Documents

PreOne document series मध्ये OpenAPI Specification चे specific position आहे. खालील documents या spec चे predecessors आहेत:

Document	Version	Relationship to OpenAPI Spec
Vision Document	v1.0	Strategic intent — API non-functional requirements (scale, security)
Master PRD	v1.0	Functional requirements — every endpoint traces to a PRD FR
DDD	v1.0	Domain model — API aggregates map to DDD aggregates
ERD	v3.0	Physical schema — API response fields map to ERD entities
API Contract Catalog	v1.0	Endpoint inventory — OpenAPI spec is the formal encoding of this catalog

2. OpenAPI Standards

2.1 Specification Version

PreOne ही OpenAPI 3.1.0 specification वापरते. 3.1 ही latest stable version आहे जी full JSON Schema 2020-12 compatibility प्रदान करते. 3.0 च्या तुलनेत 3.1 मध्ये several important improvements आहेत: nullable fields type arrays द्वारे express केले जातात (type: [string, null]), webhooks native support करते, exclusiveMinimum/exclusiveMaximum numeric values support करते, आणि const keyword support करते. हे सर्व PreOne च्या use cases साठी essential आहेत.

Version choice documented in ADR-001 (OpenAPI Version Selection). Decision rationale: 3.1 ही future-proof choice आहे कारण tooling ecosystem (Swagger UI 5+, Redocly, OpenAPI Generator 7+) mature झाला आहे. 3.0 वर राहणे म्हणजे accepting technical debt जे भविष्यात migration costlier होईल.

2.2 Standards Compliance

PreOne ची specification खालील standards strictly follow करते:

Standard	Value	Rationale
OpenAPI Version	3.1.0	Latest stable spec — full JSON Schema 2020-12 compatibility
Format	YAML primary + JSON export	YAML for human authoring; JSON for tool consumption
License	Apache 2.0	Specification text copyright-neutral; PreOne content proprietary
Schema Dialect	https://spec.openapis.org/oas/3.1/dialect/base	Base dialect allows nullable via type arrays
JSON Schema Version	2020-12	Required for OpenAPI 3.1 — supports const, exclusiveMinimum
Encoding	UTF-8	All strings UTF-8; BOM forbidden in YAML
Document Path	/specs/openapi.yaml	Monorepo path; CI validates on every commit

Standard	Value	Rationale
Validation Tool	@redocly/cli lint	Strict linting + spectral rules for PreOne conventions
Reference Style	\$ref internal first	Relative refs inside components/; remote only for shared SDK contracts
Discriminator Strategy	Discriminator on type field	Used in Webhook payloads + Integration Events

2.3 File Organization

Specification चे organization multi-file structure मध्ये आहे — single monolithic openapi.yaml file maintain करणे difficult होते जसे spec grow करते. खालील structure followed आहे:

```

specs/
├── openapi.yaml          # Root file — info, servers, security, tags, paths
(refs)
├── paths/               # Per-domain path files
│   ├── auth.yaml
│   ├── identity.yaml
│   ├── crm.yaml
│   ├── admissions.yaml
│   ├── students.yaml
│   ├── academics.yaml
│   ├── attendance.yaml
│   ├── communication.yaml
│   ├── finance.yaml
│   ├── inventory.yaml
│   ├── hr.yaml
│   ├── administration.yaml
│   ├── reports.yaml
│   └── platform.yaml
├── components/          # Reusable components
│   ├── schemas/         # Data models
│   │   ├── common/     # Pagination, Money, Error
│   │   ├── identity/   # User, Role, Permission
│   │   ├── student/    # Student, Guardian
│   │   └── ... (per domain)
│   ├── parameters/     # PageParam, SizeParam, etc.
│   ├── headers/        # X-Trace-Id, X-RateLimit-*
│   ├── responses/      # NotFound, ValidationError
│   ├── requestBodies/  # StudentCreateBody
│   ├── examples/       # Example payloads
│   ├── securitySchemes/ # BearerAuth, ApiKey
│   ├── links/          # Operation links
└── webhooks/           # Webhook event payloads

```

└─ websockets/

WebSocket channel docs

2.4 Reference Strategy

\$ref usage मध्ये consistency critical आहे. PreOne च्या convention प्रमाणे:

- Internal refs: \$ref: '#/components/schemas/Student' — root-relative
- File refs: \$ref: './components/schemas/student/Student.yaml' — relative to current file
- Remote refs: Discouraged — bundle at build time if external schema needed
- Circular refs: Allowed in components but resolved by Swagger UI at render time
- Naming: PascalCase for schemas, camelCase for fields, kebab-case for paths

3. API Information

3.1 info Object

info object हा OpenAPI spec चा first section आहे जो API चे metadata defines करतो. हे metadata Swagger UI च्या header मध्ये render होते आणि SDK generation मध्ये package metadata बनून जाते. PreOne च्या info object मध्ये standard fields beyond काही custom extensions आहेत जे internal tooling वापरते.

```
info:
  title: PreOne Enterprise API
  version: 1.0.0
  description: |
    Enterprise Preschool Operating System API.
    Multi-tenant, multi-branch, multi-academic-year.
    14 business domains, ~530 endpoints, 5 WebSocket channels, 5 webhooks.
  contact:
    name: PreOne API Team
    email: api@preone.com
    url: https://developers.preone.com
  license:
    name: PreOne Proprietary v1.0
    url: https://www.preone.com/license
  termsOfService: https://www.preone.com/terms
  x-preone-status: frozen
  x-preone-release-date: '2026-07-14'
  x-preone-deprecation-policy: https://docs.preone.com/api/deprecation
```

3.2 Field Reference

Field	Value	Description
title	PreOne Enterprise API	Public title shown in Swagger UI header
version	1.0.0	SemVer MAJOR.MINOR.PATCH — frozen for v1.0 release
description	Enterprise Preschool Operating System API — multi-tenant, multi-branch, multi-academic-year. 14 business domains, ~530 endpoints, 5 WebSocket channels, 5 webhook events.	Markdown supported; appears under title in Swagger UI

Field	Value	Description
contact.name	PreOne API Team	Email-based contact for support escalations
contact.email	api@preone.com	Auto-routes to engineering on-call
contact.url	https://developers.preone.com	Developer portal link
license.name	PreOne Proprietary v1.0	Customer license agreement references this identifier
license.url	https://www.preone.com/license	License URL
termsOfService	https://www.preone.com/terms	Terms of service URL
externalDocs.url	https://docs.preone.com/api	Pointer to long-form documentation portal

3.3 Versioning Strategy

info.version field SemVer convention follows करतो. Initial release 1.0.0 आहे. Backward-compatible additions (new endpoints, new optional fields) MINOR bump trigger करतात (1.1.0, 1.2.0...). Breaking changes MAJOR bump trigger करतात (2.0.0) आणि 12-month deprecation cycle सोबत येतात. Bug fixes आणि documentation clarifications PATCH bump trigger करतात (1.0.1, 1.0.2...).

Version history चा complete record CHANGELOG.md मध्ये maintained आहे जो release tag सोबत auto-generate होतो. प्रत्येक release सोबत Git tag (v1.0.0, v1.1.0...) आणि GitHub Release created होतो जिथे breaking changes explicitly called out असतात.

4. Servers

4.1 Server Environments

PreOne चे 4 server environments आहेत, प्रत्येकाचा specific purpose आहे. Production environment फक्त customer traffic handle करते आणि strictest controls आहेत. Staging environment प्रोडक्शन mirror आहे जिथे customer UAT आणि integration partner testing होते. QA environment internal testing साठी आहे जिथे main branch वर auto-deploy होते. Local environment developer machine वर चालते.

servers:

- url: <https://api.preone.com/api/v1>
description: Production
variables:
tenantId:
default: default
description: Override for multi-tenant routing
- url: <https://staging.preone.com/api/v1>
description: Staging
- url: <https://qa.preone.com/api/v1>
description: QA
- url: <http://localhost:3000/api/v1>
description: Local

4.2 Environment Reference

URL	Environment	Description
https://api.preone.com/api/v1	Production	Live customer traffic. TLS 1.3 enforced. Rate-limited per subscription tier.
https://staging.preone.com/api/v1	Staging	Pre-production. Mirrors prod schema. Customer UAT + integration partners.
https://qa.preone.com/api/v1	QA	Internal QA. Auto-deployed from main branch.
http://localhost:3000/api/v1	Local	Developer local. Port 3000 default NestJS port.

4.3 Environment Selection

Swagger UI मध्ये environment dropdown द्वारे select केले जाते. SDKs मध्ये environment configuration parameter द्वारे pass केले जाते. Production environment साठी additional safeguards आहेत:

- TLS 1.3 mandatory — older TLS versions rejected
- Rate limiting per subscription tier (Free: 100 req/min, Pro: 1000 req/min, Enterprise: custom)
- WAF (Web Application Firewall) blocks suspicious traffic
- Bot detection via Cloudflare
- Geo-blocking for non-India traffic (configurable per tenant)
- Real-time monitoring + alerting on 5xx rate > 1%

5. Security Schemes

5.1 Supported Schemes

PreOne authentication multi-layered आहे. Primary scheme Bearer JWT आहे जो सर्व authenticated endpoints साठी वापरला जातो. Future-ready schemes म्हणून OAuth2 (third-party partner integrations साठी), API Keys (Platform Admin आणि webhook signing साठी), आणि SSO (enterprise customers साठी IdP integration) defined आहेत. v1.0 release मध्ये फक्त Bearer JWT आणि API Key active आहेत; OAuth2 आणि SSO v1.1/v1.2 मध्ये ship होतील.

securitySchemes:

BearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

description: |

RS256-signed JWT. Obtain via POST /auth/login.

Access token TTL: 15 minutes. Refresh token TTL: 30 days.

Multi-tenant claims: tid (tenant), bid (branch), ayid (academic year),

uid (user), scopes (array of permission strings).

ApiKey:

type: apiKey

in: header

name: X-API-Key

description: |

Platform Admin + webhook signing. Per-tenant key with scoped permissions.

Rotated quarterly. Managed via POST /platform/api-keys.

OAuth2:

type: oauth2

description: Future-ready (v1.1). Partner platform integrations.

flows:

authorizationCode:

authorizationUrl: https://api.preone.com/oauth/authorize

tokenUrl: https://api.preone.com/oauth/token

scopes:

read: Read access

write: Write access

admin: Administrative access

clientCredentials:

tokenUrl: https://api.preone.com/oauth/token

scopes:

read: Read access

write: Write access

5.2 Scheme Reference

Scheme	Type	Flow / Format	Applicable Endpoints	Notes
BearerAuth	http	bearer / JWT	All authenticated endpoints	RS256-signed JWT, 15 min access + 30 d refresh rotation. Multi-tenant claims: tid, bid, ayid, uid, scopes.
OAuth2 (Future)	oauth2	authorizationCode + clientCredentials	Partner platform integrations	Roadmap v1.1 — third-party app marketplace. PKCE mandatory for SPA clients.
ApiKey (Platform)	apiKey	header: X-API-Key	Platform Admin + Partner webhooks signing	Per-tenant key with scoped permissions; rotated quarterly.
SSO (Future)	oauth2	authorizationCode	Enterprise customers with IdP	SAML/OIDC bridge. Roadmap v1.2 — Microsoft Entra, Okta, Google Workspace.

5.3 Future-Ready Authentication

OAuth2 आणि SSO हे v1.0 spec मध्ये documented आहेत पण implementation v1.1/v1.2 मध्ये ship होईल. Spec मध्ये आधीच define करणे म्हणजे SDK consumers आणि integration partners यांना future roadmap दिसते आणि त्यांनी आपली architecture accordingly plan करू शकते. OAuth2 implementation साठी PKCE mandatory राहिल SPA clients साठी; client credentials flow server-to-server integrations साठी वापरला जाईल.

6. Authentication APIs

6.1 Login Flow

PreOne चा authentication flow mobile + OTP based आहे — username/password combination नाही. हा design decision ADR-002 (Mobile OTP Authentication) मध्ये documented आहे. Rationale: preschool parents आणि teachers यांच्यासाठी password management burden कमी करणे, OTP via SMS/WhatsApp ubiquitous आहे, आणि security strong आहे कारण OTP short-lived असतो आणि every login attempt logged असतो.

Login flow दोन steps मध्ये आहे: (1) POST /auth/otp/send with mobile number — backend OTP generate करतो आणि SMS/WhatsApp पाठवतो, rate-limited per mobile (60s cooldown). (2) POST /auth/login with mobile + otp — backend OTP verify करतो आणि successful verification वर access + refresh tokens issue करतो. Step-up verification साठी POST /auth/otp/verify वापरले जाते जिथे फक्त OTP verify होतो पण tokens issue होत नाहीत.

6.2 Login Endpoint

POST /auth/login

Request Body:

```
{
  "mobile": "9876543210",
  "otp": "123456"
}
```

Response 200:

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "expiresIn": 3600,
  "tokenType": "Bearer",
  "user": {
    "id": "usr_01HXY...",
    "name": "Aarav Sharma",
    "mobile": "9876543210",
    "roles": ["teacher"],
    "permissions": ["student:read", "attendance:write"],
    "activeBranch": {
      "id": "br_01HXY...",
      "name": "Sunshine Kids - Pune"
    },
    "activeTenant": {
      "id": "ten_01HXY...",
      "name": "Sunshine Kids Group"
    }
  }
}
```

```

    }
  }

```

Response 401:

```

{
  "success": false,
  "error": {
    "code": "INVALID_OTP",
    "message": "Invalid or expired OTP"
  },
  "traceId": "abc-123-def-456"
}

```

6.3 Authentication Endpoints

Method	Path	Summary	Description	Status Codes
POST	/auth/login	Mobile + OTP login	Initiates OTP, returns access+refresh on success	200, 400, 401, 429
POST	/auth/otp/send	Send OTP via SMS/WhatsApp	Rate-limited per mobile (60s cooldown)	200, 400, 429
POST	/auth/otp/verify	Verify OTP without login	Used for step-up verification on sensitive actions	200, 400, 401
POST	/auth/refresh	Refresh access token	Rotates refresh token, invalidates old	200, 401, 403
POST	/auth/logout	Logout (revoke tokens)	Both access + refresh blacklisted; Redis TTL	200, 401
GET	/auth/me	Current user profile	Returns user + roles + permissions + active branch	200, 401

Method	Path	Summary	Description	Status Codes
POST	/auth/branch/switch	Switch active branch	Issues new access token with new bid claim	200, 401, 403
POST	/auth/password/reset/request	Password reset link	Emails reset link with signed token (15 min TTL)	200, 400
POST	/auth/password/reset/confirm	Confirm password reset	Validates token + sets new password	200, 400, 401

6.4 Token Lifecycle

Access tokens 15 minutes TTL सोबत short-lived आहेत. Refresh tokens 30 days TTL सोबत long-lived आहेत आणि rotation सोबत येतात — प्रत्येक /auth/refresh call नवीन access + refresh token issue करतो आणि old refresh token blacklist होतो. हा rotation strategy refresh token theft attack पासून protection देते. Logout केल्यावर दोन्ही tokens Redis-backed blacklist मध्ये added होतात.

- Access Token TTL: 15 minutes (900 seconds)
- Refresh Token TTL: 30 days (2,592,000 seconds)
- Refresh Token Rotation: mandatory — old token invalidated on each refresh
- Blacklist: Redis with TTL matching token expiry
- Signing Algorithm: RS256 (asymmetric, public key distributed via JWKS endpoint)
- JWKS Endpoint: GET /.well-known/jwks.json (cached for 1 hour)

7. Authorization

7.1 Scope-Based Access Control

PreOne authorization scope-based access control (SBack) वापरते जो RBAC (Role-Based Access Control) च्या वर extends होतो. Each permission एक granular scope string आहे जो format <domain>:<action> follow करतो — उदा. student:read, finance:write, platform:admin. Roles (admin, teacher, accountant, counselor) या scopes च्या collections आहेत ज्यांना tenant admin customize करू शकतो पण base scope catalog fixed आहे.

Scopes JWT मध्ये 'scopes' claim द्वारे carried असतात — array of strings. Backend each request वर scope check करतो via @RequireScopes decorator. Frontend <Can permission="student:write"> component द्वारे UI elements hide/show करते. हा dual enforcement (server + client) defense-in-depth प्रदान करतो — client-side checks UX साठी, server-side checks security साठी.

7.2 Scope Catalog

Scope	Description	Notes
identity:read	Read users, roles, permissions	Branch-scoped read; cross-branch requires platform scope
identity:write	Create/update users, assign roles	Audit logged; requires branch admin or higher
admissions:read	Read admission applications	Branch-scoped; counselor sees only assigned leads
admissions:write	Create/update admission applications	Counselor + branch admin
finance:read	Read invoices, payments, receipts	Branch-scoped; accountant role
finance:write	Create invoices, record payments, refund	Accountant + branch admin; refund requires admin
student:read	Read student records	Branch-scoped; teacher sees only own class students
student:write	Create/update student records	Branch admin; teacher can update attendance only

Scope	Description	Notes
platform:read	Platform-level read across tenants	Platform admin only — superuser scope
platform:write	Platform-level write (tenant config, billing)	Platform admin only

7.3 Authorization Flow

1. Client sends request with Authorization: Bearer <JWT>
2. Backend extracts JWT, verifies signature via JWKS
3. Backend extracts scopes claim from JWT
4. Backend checks if required scope present:
 - Route handler decorated with @RequireScopes('student:write')
 - If scope missing -> 403 FORBIDDEN
5. Backend checks tenant/branch consistency:
 - JWT tid vs X-Tenant-Id header -> mismatch = 403 TENANT_MISMATCH
 - JWT bid vs X-Branch-Id header -> mismatch = 403 BRANCH_MISMATCH
6. Backend checks ownership (resource tenant/branch = JWT tenant/branch)
 - Cross-tenant access = 404 NOT_FOUND (info leak prevention)
7. Backend executes handler, logs audit entry

Authorization Header Format:

Authorization: Bearer eyJhbGciOiJSUzI1NiIs...

Decoded JWT Payload:

```
{
  "sub": "usr_01HXY...",
  "tid": "ten_01HXY...",
  "bid": "br_01HXY...",
  "ayid": "ay_01HXY...",
  "scopes": ["student:read", "attendance:write"],
  "iat": 1736832000,
  "exp": 1736832900
}
```

8. Common Headers

8.1 Multi-Tenant Header Standards

PreOne multi-tenant, multi-branch, multi-academic-year system आहे. हेच context every API request सोबत carry होणे आवश्यक आहे. Standard headers द्वारे हे context propagate होते — याला 'Multi-Tenant Header Standards' म्हणतात. हे headers API Contract Catalog v1.0 च्या Multi-Tenant Strategy section शी पूर्णपणे aligned आहेत. JWT claims (tid, bid, ayid) आणि headers दोन्ही मध्ये same context असणे mandatory आहे — mismatch = 403 error.

Backend मध्ये TenantContext middleware हे headers parse करून AsyncLocalStorage मध्ये store करते. यामुळे downstream services (repositories, services) ला explicit parameter passing न करता context available होतो. Logging, audit trails, आणि database queries सर्व या context वापरतात automatically.

8.2 Header Reference

Header	Type	Required	Default / Source	Description
Authorization	string (JWT)	Required	Bearer <JWT>	All authenticated endpoints
X-Tenant-Id	UUID	Required	tid claim extracted from JWT	All multi-tenant endpoints
X-Branch-Id	UUID	Required	bid claim — currently active branch	Branch-scoped endpoints
X-Academic-Year	string (2025-2026)	Optional	ayid — defaults to current academic year	Academic Year-scoped endpoints
X-Timezone	string (Asia/Kolkata)	Optional	Defaults to tenant timezone	Date/time interpretation
Accept-Language	string (en, mr, hi)	Optional	Defaults to tenant default language	Localization of error messages

Header	Type	Required	Default / Source	Description
X-Trace-Id	UUID	Optional	Generated client-side; propagated to logs	Distributed tracing correlation
X-Idempotency-Key	UUID	Optional	Required on POST/PATCH for write safety	Idempotent retries
Content-Type	application/json	Required	UTF-8 encoded	All POST/PATCH with body
Accept	application/json	Required	UTF-8 encoded	All endpoints

8.3 Header Examples

```
# Full authenticated request
GET /students/stu_01HXY... HTTP/1.1
Host: api.preone.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
X-Tenant-Id: ten_01HXY...
X-Branch-Id: br_01HXY...
X-Academic-Year: 2025-2026
X-Timezone: Asia/Kolkata
Accept-Language: mr
X-Trace-Id: 550e8400-e29b-41d4-a716-446655440000
Accept: application/json

# Idempotent write request
POST /students HTTP/1.1
Host: api.preone.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
X-Tenant-Id: ten_01HXY...
X-Branch-Id: br_01HXY...
X-Idempotency-Key: 6ba7b810-9dad-11d1-80b4-00c04fd430c8
Content-Type: application/json

{
  "firstName": "Aarav",
  "lastName": "Sharma",
  "dob": "2020-05-15",
  "gender": "Male"
}
```

9. Request Schemas

9.1 Schema Standards

Request schemas JSON Schema 2020-12 draft follow करतात जे OpenAPI 3.1 चा default dialect आहे. प्रत्येक domain साठी request schemas components/schemas/{domain}/ folder मध्ये organized आहेत. Naming convention: {Aggregate}{Action}Request — उदा. StudentCreateRequest, FeeInvoiceUpdateRequest, AdmissionApplicationApproveRequest. Schemas type-safe आहेत आणि TypeScript SDK मध्ये auto-generated types बनून जातात.

9.2 Request Patterns

Pattern	HTTP	Body Shape	Response	Examples
CreateRequest	POST /resource	Required fields + optional metadata	Returns 201 + created resource	student.create, fee.invoice.create
UpdateRequest	PATCH /resource/{id}	Partial update — only changed fields	Returns 200 + updated resource	student.update, fee.invoice.update
SearchRequest	GET /resource? q=&filter=&sort=&page=&size=	Query params + pagination	Returns 200 + paginated list	student.search, fee.invoice.list
BulkRequest	POST /resource/bulk	Array of operations (max 100)	Returns 200 + per-item results	attendance.mark.bulk, fee.receipt.bulk
UploadRequest	POST /resource/upload	multipart/form-data	Returns 201 + file metadata	student.photo.upload, document.upload
ActionRequest	POST /resource/{id}/action	State machine transitions	Returns 200 + updated state	admission.approve, fee.invoice.cancel
ExportRequest	POST /resource/export	Async export with format + filter	Returns 202 + jobId; poll /jobs/{id}	reports.export, student.export

Pattern	HTTP	Body Shape	Response	Examples
WebhookTestRequest	POST /webhooks/{id}/test	Sends test event to webhook URL	Returns 200 + delivery status	platform.webhook.test

9.3 StudentCreateRequest Schema

StudentCreateRequest:

type: object

required:

- firstName
- dob
- gender

properties:

firstName:

type: string

minLength: 1

maxLength: 100

description: Student's first name (legal name)

lastName:

type: string

maxLength: 100

description: Student's last name (optional)

dob:

type: string

format: date

description: Date of birth (YYYY-MM-DD)

gender:

type: string

enum: [Male, Female, Other]

admissionNo:

type: string

pattern: '^[A-Z]{3}-\d{4}\$'

description: Auto-generated if not provided

classId:

type: string

format: uuid

description: Initial class assignment

sectionId:

type: string

format: uuid

description: Initial section assignment

guardians:

type: array

maxItems: 4

items:

\$ref: '#/components/schemas/GuardianCreateRequest'

address:

\$ref: '#/components/schemas/AddressRequest'

additionalProperties: false

example:

firstName: Aarav

lastName: Sharma

dob: '2020-05-15'

gender: Male

classId: cls_01HXY...

sectionId: sec_01HXY...

guardians:

- name: Rohan Sharma

relation: Father

mobile: '9876543210'

10. Response Schemas

10.1 Response Standards

Response schemas strict typing follow करतात. प्रत्येक endpoint चे response status code-specific schema define केलेले आहे — उदा. 200 OK चे schema, 201 Created चे schema, 400 Validation Error चे schema सर्व separate आहेत. हे approach SDK generation मध्ये critical आहे कारण generated clients type-narrow करू शकतात based on status code. PreOne च्या convention प्रमाणे response objects envelope मध्ये wrapped नाहीत — direct resource object return होतो. Errors साठी मात्र standard envelope आहे.

10.2 Response Patterns

Pattern	Status	Body Shape	Example Endpoints
SingleResource	200 OK	Resource object with id + metadata	GET /students/{id}
CreatedResource	201 Created	Newly created resource with id + createdAt	POST /students
UpdatedResource	200 OK	Updated resource with version + updatedAt	PATCH /students/{id}
DeletedResource	204 No Content	Empty body	DELETE /students/{id}
CollectionResponse	200 OK	Paginated array + meta (total, page, size, hasNext)	GET /students
AcceptedJob	202 Accepted	jobId + status=pending + estimated completion	POST /students/export
BulkResponse	200 OK	Array of per-item results (success/failure + error)	POST /attendance/bulk

Pattern	Status	Body Shape	Example Endpoints
HealthCheck	200 OK	status + uptime + version + dependency status	GET /health

10.3 StudentResponse Schema

```

StudentResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
      example: stu_01HXY...
    admissionNo:
      type: string
      example: SKP-0001
    firstName:
      type: string
      example: Aarav
    lastName:
      type: string
      example: Sharma
    dob:
      type: string
      format: date
      example: '2020-05-15'
    gender:
      type: string
      enum: [Male, Female, Other]
    class:
      $ref: '#/components/schemas/ClassReference'
    section:
      $ref: '#/components/schemas/SectionReference'
    guardians:
      type: array
      items:
        $ref: '#/components/schemas/GuardianResponse'
    status:
      type: string
      enum: [Active, Inactive, Graduated, Transferred]
    createdAt:
      type: string
      format: date-time
    updatedAt:
      type: string
      format: date-time
    version:
      type: integer

```

description: Optimistic locking version
 required: [id, admissionNo, firstName, dob, gender, status, createdAt, updatedAt, version]

10.4 Pagination Response

List endpoints standard pagination response return करतात. Page-based pagination default आहे (page + size); cursor-based pagination available आहे very large datasets साठी via ? cursor= parameter. Page size maximum 100 आहे — larger requests rejected with VALIDATION_ERROR.

```
PaginatedStudents:
  type: object
  properties:
    data:
      type: array
      items:
        $ref: '#/components/schemas/StudentResponse'
    meta:
      type: object
      properties:
        page:
          type: integer
          minimum: 1
        size:
          type: integer
          minimum: 1
          maximum: 100
        total:
          type: integer
          minimum: 0
        totalPages:
          type: integer
          minimum: 0
        hasNext:
          type: boolean
        hasPrev:
          type: boolean
      required: [page, size, total, totalPages, hasNext, hasPrev]
  required: [data, meta]
example:
  data:
    - id: stu_01HXY...
      admissionNo: SKP-0001
      firstName: Aarav
      lastName: Sharma
  meta:
    page: 2
    size: 20
    total: 145
```

totalPages: 8
hasNext: true
hasPrev: true

11. Error Schemas

11.1 Standard Error Envelope

Errors साठी एक standard envelope आहे जे सर्व endpoints साठी consistent आहे. हा envelope success responses पासून completely different structure मध्ये असतो — client SDKs type-narrow करू शकतात 'success' boolean field द्वारे. Error envelope मध्ये three primary components आहेत: success (always false), error (object with code + message + optional field-level details), आणि traceId (for support correlation).

```
Error:
  type: object
  properties:
    success:
      type: boolean
      enum: [false]
    error:
      type: object
      properties:
        code:
          type: string
          description: Machine-readable error code
          example: STUDENT_NOT_FOUND
        message:
          type: string
          description: Human-readable error message
          example: Student not found
        details:
          type: array
          description: Field-level validation errors (only for 400)
          items:
            type: object
            properties:
              field:
                type: string
                example: firstName
              message:
                type: string
                example: firstName is required
              code:
                type: string
                example: REQUIRED_FIELD_MISSING
            required: [code, message]
      traceId:
        type: string
        format: uuid
        description: Correlation ID for support tickets
        example: 550e8400-e29b-41d4-a716-446655440000
```

required: [success, error, traceId]

Example Error Response:

```
{
  "success": false,
  "error": {
    "code": "STUDENT_NOT_FOUND",
    "message": "Student not found"
  },
  "traceId": "abc123"
}
```

11.2 Common Status Codes

HTTP	Meaning	When Used
200	Success	Standard successful GET, PATCH
201	Created	Successful POST creating new resource
202	Accepted	Async job accepted (export, bulk operation)
204	No Content	Successful DELETE, no body to return
400	Validation Error	Request body/params failed schema validation
401	Unauthorized	Missing/expired JWT
403	Forbidden	Insufficient scope or tenant/branch mismatch
404	Not Found	Resource does not exist or not in tenant scope
409	Conflict	Duplicate resource or version conflict
422	Business Rule Violation	Domain invariant violated (e.g., student already admitted)
429	Rate Limited	Too many requests; Retry-After header returned

HTTP	Meaning	When Used
500	Internal Server Error	Unhandled server error; logged with traceId
503	Service Unavailable	Downstream dependency failure

11.3 Error Code Catalog

HTTP	Code	Description	Notes
400	VALIDATION_ERROR	Request body failed schema validation	Returns field-level errors array
400	INVALID_QUERY_PARAMETER	Query parameter invalid	Returns param + reason
401	UNAUTHENTICATED	Missing or invalid JWT	Client should re-authenticate
401	TOKEN_EXPIRED	Access token expired	Client should call /auth/refresh
403	FORBIDDEN	Insufficient scope/role	User authenticated but lacks permission
403	TENANT_MISMATCH	Header tid ≠ JWT tid	Cross-tenant access attempt
404	NOT_FOUND	Resource not found	Resource id does not exist or tenant-scoped
404	ROUTE_NOT_FOUND	Endpoint not found	Path doesn't match any route
409	CONFLICT	State conflict	E.g., duplicate admission number, version mismatch
409	VERSION_MISMATCH	Optimistic lock failed	Resource updated by another transaction
422	BUSINESS_RULE_VIOLATION	Domain invariant violated	E.g., student already admitted, invoice already paid

HTTP	Code	Description	Notes
429	RATE_LIMIT_EXCEEDED	Too many requests	Returns Retry-After header
500	INTERNAL_ERROR	Unhandled server error	Logged with traceId; user sees generic message
503	SERVICE_UNAVAILABLE	Downstream dependency failure	E.g., DB, Redis, SMS gateway down

12. Common Components

12.1 Components Overview

OpenAPI components/ section हा reusability का केंद्र आहे. Common schemas, parameters, headers, responses, examples, आणि security schemes सर्व येथे defined आहेत जे \$ref द्वारे across multiple paths reused होतात. हे approach spec चे size कमी करते, consistency सुनिश्चित करते, आणि maintenance सोपी करते — एक change केल्यावर सर्व consumers automatic update देखाली येतात.

PreOne च्या components/ मध्ये 9 sub-folders आहेत: schemas (data models), parameters (reusable path/query/header params), headers (response headers), responses (reusable response objects), requestBodies (reusable request payloads), securitySchemes (auth schemes), examples (example payloads), links (operation links), आणि callbacks (webhook contracts). प्रत्येक sub-folder per-domain organized आहे जिथे applicable.

12.2 Component Reference

Folder	Purpose	Examples	Usage
schemas/	Reusable data models	Student, Guardian, Address, Money, Pagination, Error	Referenced by \$ref: '#/components/schemas/Student'
parameters/	Reusable path/query/header parameters	PageParam, SizeParam, SortParam, TenantIdHeader	Referenced by \$ref: '#/components/parameters/PageParam'
headers/	Reusable response headers	X-Trace-Id, X-RateLimit-Limit, X-RateLimit-Remaining	Referenced by \$ref: '#/components/headers/XTraceId'
responses/	Reusable response objects	NotFound, ValidationError, Unauthorized, Forbidden	Referenced by \$ref: '#/components/responses/NotFound'
requestBodies/	Reusable request bodies	StudentCreateBody, BulkAttendanceBody	Referenced by \$ref: '#/components/requestBodies/StudentCreateBody'

Folder	Purpose	Examples	Usage
securitySchemes/	Auth schemes	BearerAuth, ApiKey	Referenced at operation or root level
examples/	Reusable example payloads	StudentCreateExample, ValidationErrorMessage	Referenced by \$ref: '#/components/examples/StudentCreateExample'
links/	Operation links	CreateStudent → GetStudentById link	Enables Swagger UI 'try it out' chaining
callbacks/	Webhook callbacks	StudentAdmitted	Referenced in operation callbacks

12.3 Reusable Parameter Example

```

components:
  parameters:
    PageParam:
      name: page
      in: query
      description: Page number (1-indexed)
      required: false
      schema:
        type: integer
        minimum: 1
        default: 1
    SizeParam:
      name: size
      in: query
      description: Page size (max 100)
      required: false
      schema:
        type: integer
        minimum: 1
        maximum: 100
        default: 20
    SortParam:
      name: sort
      in: query
      description: |
        Sort fields. Prefix with - for descending.
        Example: -createdAt,firstName
      required: false
      schema:
        type: string
        pattern: '^-[a-zA-Z]+(,-?[a-zA-Z]+)*$'

```

```
TenantIdHeader:
  name: X-Tenant-Id
  in: header
  required: true
  schema:
    type: string
    format: uuid
```

```
# Usage in path:
/students:
  get:
    parameters:
      - $ref: '#/components/parameters/PageParam'
      - $ref: '#/components/parameters/SizeParam'
      - $ref: '#/components/parameters/SortParam'
      - $ref: '#/components/parameters/TenantIdHeader'
```

12.4 Reusable Response Example

```
components:
  responses:
    NotFound:
      description: Resource not found
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
          example:
            success: false
            error:
              code: NOT_FOUND
              message: Resource not found
              traceId: 550e8400-e29b-41d4-a716-446655440000
    ValidationError:
      description: Request validation failed
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
          example:
            success: false
            error:
              code: VALIDATION_ERROR
              message: Request validation failed
              details:
                - field: firstName
                  message: firstName is required
                  code: REQUIRED_FIELD_MISSING
              traceId: 550e8400-e29b-41d4-a716-446655440000
```


13. Identity APIs

13.1 Identity Domain Overview

Identity domain हा PreOne चा foundational domain आहे जो users, roles, permissions, आणि session management handle करतो. हा domain सर्व other domains चा prerequisite आहे — कोणताही authenticated API call Identity domain द्वारे issued JWT ने येतो. Identity endpoints 'Identity' tag अंतर्गत grouped आहेत आणि /identity/ prefix वापरतात.

Identity domain चे key aggregates: User (root aggregate with credentials + profile + roles), Role (collection of permissions), Permission (granular scope string), Session (active JWT sessions). Role-Permission association many-to-many आहे; User-Role association many-to-many per-branch आहे (user एका branch मध्ये teacher, दुसऱ्या branch मध्ये admin असू शकतो).

13.2 Identity Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/identity/users	Create user (admin/teacher /staff)	identity:write	201, 400, 409
GET	/identity/users	List users with pagination	identity:read	200, 400
GET	/identity/ users/{id}	Get user by id	identity:read	200, 404
PATCH	/identity/ users/{id}	Update user	identity:write	200, 404, 409
DELETE	/identity/ users/{id}	Soft-delete user	identity:write	204, 404
POST	/identity/roles	Create role	identity:write	201, 400, 409
GET	/identity/roles	List roles	identity:read	200
POST	/identity/ permissions	Create custom permission	identity:write	201, 400, 409
POST	/identity/ users/{id}/roles	Assign roles to user	identity:write	200, 404, 409
POST	/identity/ sessions/revoke	Revoke user sessions	identity:write	200, 404

13.3 Create User Example

POST /identity/users

Request:

```
{
  "name": "Aarav Sharma",
  "mobile": "9876543210",
  "email": "aarav@example.com",
  "roles": ["teacher"],
  "branchId": "br_01HXY..."
}
```

Response 201:

```
{
  "id": "usr_01HXY...",
  "name": "Aarav Sharma",
  "mobile": "9876543210",
  "email": "aarav@example.com",
  "roles": [
    {
      "id": "rol_teacher",
      "name": "Teacher",
      "scopes": ["student:read", "attendance:write"]
    }
  ],
  "branchId": "br_01HXY...",
  "status": "Active",
  "createdAt": "2026-07-14T10:30:00Z"
}
```

Response 409 (CONFLICT):

```
{
  "success": false,
  "error": {
    "code": "USER_ALREADY_EXISTS",
    "message": "User with this mobile number already exists"
  },
  "traceId": "abc-123"
}
```

14. CRM APIs

14.1 CRM Domain Overview

CRM (Customer Relationship Management) domain हा admissions funnel चा top portion handle करतो. Leads capture, follow-ups, campaigns, आणि lead-to-application conversion या सर्व workflows येथून सुरू होतात. CRM endpoints 'CRM' tag अंतर्गत grouped आहेत आणि /crm/ prefix वापरतात. Key aggregates: Lead (with status state machine), Campaign (lead source attribution), FollowUp (interaction history).

Lead status state machine: New → Contacted → Interested → Visit Scheduled → Application Submitted → Won/Lost. हे transitions server-side enforced आहेत — invalid transitions 422 BUSINESS_RULE_VIOLATION return करतात. Lead conversion (/crm/leads/{id}/convert) हा critical endpoint आहे जो Lead aggregate ला AdmissionApplication aggregate मध्ये transform करतो आणि cross-domain event publish करतो.

14.2 CRM Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/crm/leads	Create lead	crm:write	201, 400
GET	/crm/leads	List leads with filters	crm:read	200, 400
GET	/crm/leads/{id}	Get lead details	crm:read	200, 404
PATCH	/crm/leads/{id}	Update lead	crm:write	200, 404, 409
DELETE	/crm/leads/{id}	Soft-delete lead	crm:write	204, 404
POST	/crm/leads/{id}/followups	Add follow-up	crm:write	201, 404
POST	/crm/leads/{id}/convert	Convert lead to admission application	crm:write	200, 404, 422
GET	/crm/campaigns	List campaigns	crm:read	200
POST	/crm/campaigns	Create campaign	crm:write	201, 400
GET	/crm/analytics/funnel	Lead funnel analytics	crm:read	200

14.3 Lead Conversion Example

POST /crm/leads/lead_01HXY.../convert

Request:

```
{
  "applicationData": {
    "academicYearId": "ay_01HXY...",
    "classId": "cls_01HXY...",
    "documents": ["doc_01HXY..."]
  }
}
```

Response 200:

```
{
  "leadId": "lead_01HXY...",
  "applicationId": "app_01HXY...",
  "applicationStatus": "Submitted",
  "convertedAt": "2026-07-14T11:00:00Z"
}
```

Response 422 (BUSINESS_RULE_VIOLATION):

```
{
  "success": false,
  "error": {
    "code": "LEAD_NOT_CONVERTIBLE",
    "message": "Lead must be in 'Interested' or 'Visit Scheduled' status to convert"
  },
  "traceId": "abc-456"
}
```

15. Admission APIs

15.1 Admission Domain Overview

Admission domain हा CRM च्या पाठीशी येतो आणि application review + approval workflow handle करतो. Application lifecycle: Submitted → Document Verification → Interview Scheduled → Interview Completed → Approved/Rejected → Student Record Created. हा workflow state machine enforced आहे आणि प्रत्येक transition वर specific permissions + audit logging required आहे. Application approve झाल्यावर Student record automatically create होतो आणि StudentAdmitted webhook fire होतो.

Admission domain चे key aggregates: AdmissionApplication (root), ApplicationDocument (child entity), Interview (child entity), DocumentVerification (child entity). Application approve हा saga pattern द्वारे orchestrate होतो: Application → Student created → Initial Invoice generated → Welcome communication sent. हे सर्व transactional outbox pattern द्वारे reliable आहे — काही fail झाल्यास compensation logic rollback करतो.

15.2 Admission Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/admissions/applications	Submit admission application	admissions:write	201, 400, 409
GET	/admissions/applications	List applications with filters	admissions:read	200, 400
GET	/admissions/applications/{id}	Get application details	admissions:read	200, 404
PATCH	/admissions/applications/{id}	Update application	admissions:write	200, 404, 409
POST	/admissions/applications/{id}/approve	Approve application	admissions:approve	200, 404, 422
POST	/admissions/applications/{id}/reject	Reject application	admissions:approve	200, 404, 422

Method	Path	Summary	Required Scope	Status Codes
POST	/admissions/applications/{id}/documents	Upload application document	admissions:write	201, 404
POST	/admissions/applications/{id}/interviews	Schedule interview	admissions:write	201, 404
GET	/admissions/dashboard	Admission dashboard metrics	admissions:read	200
POST	/admissions/applications/bulk-import	Bulk import applications	admissions:write	202, 400

15.3 Approve Application Example

POST /admissions/applications/app_01HXY.../approve

Request:

```
{
  "comments": "All documents verified. Approved for LKG-A.",
  "classId": "cls_01HXY...",
  "sectionId": "sec_01HXY...",
  "feeStructureId": "fs_01HXY..."
}
```

Response 200:

```
{
  "applicationId": "app_01HXY...",
  "status": "Approved",
  "approvedBy": "usr_01HXY...",
  "approvedAt": "2026-07-14T12:00:00Z",
  "studentId": "stu_01HXY...",
  "invoiceId": "inv_01HXY...",
  "webhookEventId": "evt_01HXY..."
}
```

16. Student APIs

16.1 Student Domain Overview

Student domain हा PreOne चा core domain आहे जो student records चा master data manage करतो. Student aggregate मध्ये student profile, guardians (max 4), address, academic history, attendance history, fee history — सर्व included आहे. हा aggregate eventually consistent आहे — updates propagate to derived views (reports, dashboards) via domain events. Student endpoints standard CRUD pattern follow करतात.

Student soft-deletion supported आहे — DELETE request record permanently remove करत नाही, फक्त deletedAt timestamp set करतो. हे data integrity साठी essential आहे कारण students historical reports मध्ये reference होत असतात. Hard delete फक्त platform admin via separate endpoint करू शकतो जो 90-day grace period सोबत येतो.

16.2 CRUD Pattern

हेच pattern Admissions, Finance, HR, Inventory, Communication आणि इतर सर्व domains साठी लागू होईल. Standard CRUD endpoints:

```
GET    /students          # List with pagination + filters
GET    /students/{id}     # Get by id
POST   /students          # Create new
PATCH /students/{id}     # Update existing
DELETE /students/{id}     # Soft delete
```

Additional endpoints:

```
GET    /students/{id}/guardians # Sub-resource list
POST   /students/{id}/guardians # Sub-resource create
GET    /students/{id}/attendance # Cross-domain view
GET    /students/{id}/fees       # Cross-domain view
POST   /students/export          # Async export
```

16.3 Student Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/students	Create student record	student:write	201, 400, 409
GET	/students	List students with pagination + filters	student:read	200, 400

Method	Path	Summary	Required Scope	Status Codes
GET	/students/{id}	Get student by id	student:read	200, 404
PATCH	/students/{id}	Update student	student:write	200, 404, 409
DELETE	/students/{id}	Soft-delete student	student:write	204, 404
GET	/students/{id}/guardians	List guardians	student:read	200, 404
POST	/students/{id}/guardians	Add guardian	student:write	201, 404, 409
GET	/students/{id}/attendance	Attendance history	student:read	200, 404
GET	/students/{id}/fees	Fee history	student:read	200, 404
POST	/students/export	Async export to Excel	student:read	202, 400

16.4 Student Example

POST /students

Request:

```
{
  "firstName": "Aarav",
  "lastName": "Sharma",
  "dob": "2020-05-15",
  "gender": "Male",
  "classId": "cls_01HXY...",
  "sectionId": "sec_01HXY..."
}
```

Response 201:

```
{
  "id": "stu_01HXY...",
  "admissionNo": "SKP-0001",
  "firstName": "Aarav",
  "lastName": "Sharma",
  "dob": "2020-05-15",
  "gender": "Male",
  "status": "Active",
  "createdAt": "2026-07-14T10:00:00Z",
  "updatedAt": "2026-07-14T10:00:00Z",
}
```

```
"version": 1  
}
```

17. Academics APIs

17.1 Academics Domain Overview

Academics domain हा Class → Section → Subject → Timetable → Exam → Marks hierarchy manage करतो. Class हा top-level entity आहे (e.g., LKG, UKG, Nursery, Playgroup). प्रत्येक Class चे multiple Sections असतात (e.g., LKG-A, LKG-B). Subjects Class-स्तरावर defined असतात. Timetable Section-स्तरावर weekly schedule defines करतो. Exams Class-स्तरावर defined असतात आणि Marks Student-स्तरावर entered होतात.

Academics domain चे key entities: Class, Section, Subject, Timetable, TimetableSlot, Exam, Mark. Hierarchy strict आहे — Section Class शिवाय exist करू शकत नाही, TimetableSlot Timetable शिवाय exist करू शकत नाही. Bulk operations supported आहेत — bulk timetable generation, bulk marks entry, bulk exam creation. Async jobs द्वारे handled होतात जे 202 Accepted return करतात.

17.2 Academics Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/academics/classes	Create class	academics:write	201, 400
GET	/academics/classes	List classes	academics:read	200
POST	/academics/sections	Create section	academics:write	201, 400
GET	/academics/sections	List sections	academics:read	200
POST	/academics/subjects	Create subject	academics:write	201, 400
GET	/academics/subjects	List subjects	academics:read	200
POST	/academics/timetable	Create timetable slot	academics:write	201, 400, 409
GET	/academics/timetable	Get timetable for class/section	academics:read	200
POST	/academics/exams	Create exam	academics:write	201, 400

Method	Path	Summary	Required Scope	Status Codes
POST	/academics/marks/bulk	Bulk enter marks	academics:write	202, 400

17.3 Bulk Marks Entry Example

POST /academics/marks/bulk

Request:

```
{
  "examId": "exm_01HXY...",
  "subjectId": "sub_01HXY...",
  "marks": [
    {
      "studentId": "stu_01HXY...",
      "marksObtained": 85,
      "remarks": "Good performance"
    },
    {
      "studentId": "stu_02HXY...",
      "marksObtained": 92,
      "remarks": "Excellent"
    }
  ]
}
```

Response 202 (Accepted):

```
{
  "jobId": "job_01HXY...",
  "status": "pending",
  "estimatedCompletionTime": "2026-07-14T10:01:00Z"
}
```

Poll job status:

GET /reports/jobs/job_01HXY...

Response 200:

```
{
  "jobId": "job_01HXY...",
  "status": "completed",
  "progress": 100,
  "result": {
    "totalProcessed": 2,
    "successCount": 2,
    "failureCount": 0
  }
}
```

18. Attendance APIs

18.1 Attendance Domain Overview

Attendance domain हा daily preschool operations चा critical function आहे. Two types of attendance supported: Class Attendance (teacher marks students present/absent) आणि Transport Attendance (vehicle staff marks students boarded/alighted). Class attendance per-day per-student one record आहे; Transport attendance per-trip per-student one record आहे. Both types WebSocket channels द्वारे real-time updates push करतात — attendance.live आणि transport.live.

Attendance marking stateful operation आहे — already marked attendance update करण्यासाठी Correction Request workflow आहे. Teacher directly edit करू शकत नाही; correction request create करतो जो Branch Admin approve करतो. हे audit trail साठी essential आहे. Bulk attendance endpoint class-स्तरावर सर्व students एकाच वेळी mark करण्यासाठी आहे आणि async job द्वारे process होतो.

18.2 Attendance Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/attendance/mark	Mark single student attendance	attendance:write	201, 400, 422
POST	/attendance/mark/bulk	Bulk mark attendance for class	attendance:write	202, 400
GET	/attendance/students/{id}	Get attendance for student	attendance:read	200, 404
GET	/attendance/class/{classId}	Get attendance for class	attendance:read	200
GET	/attendance/summary	Attendance summary (date range)	attendance:read	200, 400
POST	/attendance/transport	Mark transport attendance	attendance:write	201, 400

Method	Path	Summary	Required Scope	Status Codes
GET	/attendance/transport/live	Live transport attendance stream	attendance:read	200
POST	/attendance/corrections	Request attendance correction	attendance:write	201, 400
PATCH	/attendance/corrections/{id}/approve	Approve correction	attendance:approve	200, 404, 422
GET	/attendance/reports/monthly	Monthly attendance report	attendance:read	200, 400

18.3 Bulk Attendance Example

POST /attendance/mark/bulk

Request:

```
{
  "date": "2026-07-14",
  "classId": "cls_01HXY...",
  "sectionId": "sec_01HXY...",
  "records": [
    {
      "studentId": "stu_01HXY...",
      "status": "Present"
    },
    {
      "studentId": "stu_02HXY...",
      "status": "Absent",
      "reason": "Sick leave"
    }
  ]
}
```

Response 202 (Accepted):

```
{
  "jobId": "job_att_01HXY...",
  "status": "pending"
}
```

```
# WebSocket notification when complete:
# Channel: attendance.live
# Room: branch:br_01HXY...
```

```
{
  "event": "attendance.marked",
  "payload": {
    "studentIds": ["stu_01HXY...", "stu_02HXY..."],
    "date": "2026-07-14",
    "markedBy": "usr_01HXY...",
    "markedAt": "2026-07-14T09:30:00Z"
  }
}
```

19. Communication APIs

19.1 Communication Domain Overview

Communication domain हा omnichannel messaging manage करतो — SMS, WhatsApp, Email, In-App Notifications, आणि real-time Chat. हे सर्व channels unified interface द्वारे exposed आहेत. Templates centralised आहेत जिथे variable placeholders आहेत (e.g., 'Dear {{parentName}}, your child {{studentName}} was marked absent today.'). Templates approved होणे आवश्यक आहे त्यानंतरच usage साठी available होतात — यामुळे brand consistency आणि compliance (SMS/WhatsApp DLT regulations) सुनिश्चित होते.

Communication domain चे key aggregates: Message (single outbound message with channel + template + recipient + status), Template (reusable message body with variables), Group (recipient collection for broadcast), ChatRoom (real-time 1:1 or group chat), Notification (in-app notification with read/unread state). Message sending async operation आहे — 202 Accepted return होतो आणि actual delivery background job द्वारे होतो. Delivery status update webhook द्वारे available आहे.

19.2 Communication Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/communication/messages	Send message (SMS/WhatsApp /Email)	communication: write	202, 400
GET	/communication/messages	List messages	communication: read	200, 400
POST	/communication/templates	Create template	communication: write	201, 400, 409
GET	/communication/templates	List templates	communication: read	200
POST	/communication/groups	Create communication group	communication: write	201, 400

Method	Path	Summary	Required Scope	Status Codes
POST	/communication/broadcast	Broadcast to group	communication: write	202, 400
GET	/communication/chat/rooms	List chat rooms	communication: read	200
POST	/communication/chat/messages	Send chat message	communication: write	201, 400
GET	/communication/notifications	List user notifications	communication: read	200
POST	/communication/notifications/{id}/read	Mark notification read	communication: write	200, 404

19.3 Send Message Example

POST /communication/messages

Request:

```
{
  "channel": "WhatsApp",
  "templateId": "tpl_absent_alert",
  "recipients": [
    {
      "mobile": "9876543210",
      "variables": {
        "parentName": "Rohan Sharma",
        "studentName": "Aarav Sharma"
      }
    }
  ]
}
```

Response 202 (Accepted):

```
{
  "messageId": "msg_01HXY...",
  "status": "queued",
  "channel": "WhatsApp",
  "recipientCount": 1,
  "queuedAt": "2026-07-14T10:30:00Z"
}
```

}

20. Finance APIs

20.1 Finance Domain Overview

Finance domain हा financial transactions manage करतो — invoices, payments, receipts, refunds. Invoice lifecycle: Draft → Issued → Partially Paid → Paid → Cancelled. Draft state मध्ये invoice edit करणे allowed आहे; Issued state नंतर फक्त payment receive करणे किंवा cancel करणे possible आहे. Payments दोन प्रकारचे: Online (Razorpay/PayU gateway integration) आणि Offline (cash/cheque/UPI manual entry). Refunds Online payments साठी only allowed आहेत आणि original payment amount च्या 50% पर्यंत limit आहे.

Finance domain चे key aggregates: Invoice (root with line items), Payment (recorded payment with method + reference), Receipt (auto-generated on payment success), Refund (linked to original Payment). Financial immutability principle: once Issued, invoice never modified — corrections via credit notes. हे accounting standards (GAAP/IndAS) compliance साठी essential आहे.

20.2 Finance Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/finance/ invoices	Create invoice	finance:write	201, 400, 422
GET	/finance/ invoices	List invoices	finance:read	200, 400
GET	/finance/ invoices/{id}	Get invoice	finance:read	200, 404
PATCH	/finance/ invoices/{id}	Update invoice (draft only)	finance:write	200, 404, 422
POST	/finance/ invoices/{id}/ cancel	Cancel invoice	finance:write	200, 404, 422
POST	/finance/ payments	Record payment	finance:write	201, 400, 422
POST	/finance/ payments/ online	Initiate online payment	finance:write	200, 400

Method	Path	Summary	Required Scope	Status Codes
POST	/finance/ payments/{id}/ refund	Refund payment	finance:write	200, 404, 422
GET	/finance/ receipts	List receipts	finance:read	200, 400
GET	/finance/ reports/ summary	Finance summary report	finance:read	200, 400

20.3 Create Invoice Example

POST /finance/invoices

Request:

```
{
  "studentId": "stu_01HXY...",
  "academicYearId": "ay_01HXY...",
  "invoiceType": "Tuition Fee",
  "dueDate": "2026-08-15",
  "lineItems": [
    {
      "description": "Tuition Fee - Q1 2025-2026",
      "amount": 15000.00,
      "tax": 0,
      "discount": 0
    }
  ],
  "notes": "Pay before due date to avoid late fee"
}
```

Response 201:

```
{
  "id": "inv_01HXY...",
  "invoiceNo": "INV-2026-0001",
  "studentId": "stu_01HXY...",
  "status": "Issued",
  "totalAmount": 15000.00,
  "dueDate": "2026-08-15",
  "createdAt": "2026-07-14T11:00:00Z",
  "version": 1
}
```

21. Inventory APIs

21.1 Inventory Domain Overview

Inventory domain हा physical assets manage करतो — stationery, books, uniforms, snacks, teaching materials. Stock movements हे core concept आहेत: Stock-In (receive from supplier), Stock-Out (issue to class/teacher), Adjustment (loss/damage correction), Transfer (between branches). प्रत्येक movement audit logged आहे आणि stock levels auto-update होतात. Purchase Orders supplier procurement handle करतात — PO lifecycle: Draft → Submitted → Approved → Partially Received → Fully Received → Closed.

Inventory domain चे key aggregates: Item (with category + unit + current stock level), Category (classification), StockMovement (immutable record of every movement), PurchaseOrder (procurement document with line items). Stock levels per-branch tracked आहेत — cross-branch stock queries require platform:read scope. Low stock alerts automatic होतात via background job जो threshold check करतो आणि communication domain ला alert पाठवतो.

21.2 Inventory Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/inventory/items	Create item	inventory:write	201, 400
GET	/inventory/items	List items	inventory:read	200, 400
POST	/inventory/categories	Create category	inventory:write	201, 400
POST	/inventory/stock/in	Stock-in (receive)	inventory:write	201, 400, 422
POST	/inventory/stock/out	Stock-out (issue)	inventory:write	201, 400, 422
GET	/inventory/stock/levels	Current stock levels	inventory:read	200
POST	/inventory/purchase-orders	Create purchase order	inventory:write	201, 400

Method	Path	Summary	Required Scope	Status Codes
PATCH	/inventory/ purchase- orders/{id}/ receive	Receive PO items	inventory:write	200, 404, 422
GET	/inventory/ audit	Stock audit log	inventory:read	200
POST	/inventory/ stock/adjust	Stock adjustment (loss/damage)	inventory:write	201, 400, 422

21.3 Stock Movement Example

POST /inventory/stock/out

Request:

```
{
  "itemId": "itm_01HXY...",
  "quantity": 50,
  "reason": "Issued to LKG-A class",
  "recipientId": "cls_01HXY...",
  "reference": "ISSUE-2026-0001"
}
```

Response 201:

```
{
  "movementId": "mov_01HXY...",
  "itemId": "itm_01HXY...",
  "movementType": "StockOut",
  "quantity": 50,
  "remainingStock": 450,
  "movedAt": "2026-07-14T12:00:00Z",
  "movedBy": "usr_01HXY..."
}
```

22. HR APIs

22.1 HR Domain Overview

HR domain हा employee lifecycle manage करतो — recruitment (out of scope), onboarding, leave management, payroll, attendance, offboarding. Employee आणि User यांचा linkage 1:1 आहे — प्रत्येक Employee चा एक User account असतो जो login साठी वापरला जातो. Roles assigned via User account. Employee types: Teaching Staff (with subject + class assignment), Non-Teaching Staff (administrative roles), Contract Staff (hourly). Payroll monthly run द्वारे processed होतो आणि payslips generated होतात.

HR domain चे key aggregates: Employee (root with personal + employment details), Leave (with type + duration + status + approver), Payslip (monthly salary computation with earnings + deductions), EmployeeAttendance (different from Student Attendance). Leave workflow: Applied → Pending → Approved/Rejected → Cancelled. Multi-level approval आवश्यक असल्यास sequence द्वारे routed होतो.

22.2 HR Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/hr/employees	Create employee	hr:write	201, 400, 409
GET	/hr/employees	List employees	hr:read	200, 400
GET	/hr/employees/{id}	Get employee	hr:read	200, 404
PATCH	/hr/employees/{id}	Update employee	hr:write	200, 404, 409
POST	/hr/leaves	Apply for leave	hr:write	201, 400, 422
PATCH	/hr/leaves/{id}/approve	Approve leave	hr:approve	200, 404, 422
POST	/hr/payroll/run	Run payroll for month	hr:write	202, 400
GET	/hr/payroll/{id}	Get payslip	hr:read	200, 404
POST	/hr/attendance	Mark employee attendance	hr:write	201, 400

Method	Path	Summary	Required Scope	Status Codes
GET	/hr/reports/headcount	Headcount report	hr:read	200

22.3 Apply Leave Example

POST /hr/leaves

Request:

```
{
  "employeeId": "emp_01HXY...",
  "leaveType": "Casual Leave",
  "startDate": "2026-08-10",
  "endDate": "2026-08-12",
  "reason": "Personal work",
  "approverId": "usr_02HXY..."
}
```

Response 201:

```
{
  "id": "lv_01HXY...",
  "employeeId": "emp_01HXY...",
  "status": "Pending",
  "days": 3,
  "appliedAt": "2026-07-14T13:00:00Z"
}
```

23. Administration APIs

23.1 Administration Domain Overview

Administration domain हा tenant-level configuration manage करतो — branches, academic years, holidays, audit logs, system health, maintenance mode. हे superuser-level operations आहेत जे फक्त Branch Admin किंवा Platform Admin role वाले users perform करू शकतात. Branch creation हा critical operation आहे जो multiple aggregates initialize करतो — default roles, default fee structures, default academic calendar. Audit log query read-only आहे पण powerful filter + export features देतो.

Administration domain चे key aggregates: Branch (location with address + contact + config), AcademicYear (yearly period with start/end dates + terms), Holiday (institutional holiday calendar), AuditLog (immutable event log with user + action + entity + before/after diff). Maintenance mode हा feature flag आहे जो enable केल्यावर non-admin users ला 503 Service Unavailable return करतो आणि maintenance page दिसतो.

23.2 Administration Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/administration/branches	Create branch	admin:write	201, 400, 409
GET	/administration/branches	List branches	admin:read	200
PATCH	/administration/branches/{id}	Update branch	admin:write	200, 404
POST	/administration/academic-years	Create academic year	admin:write	201, 400, 409
POST	/administration/holidays	Declare holiday	admin:write	201, 400
GET	/administration/audit-logs	Query audit logs	admin:read	200, 400

Method	Path	Summary	Required Scope	Status Codes
POST	/administration/backup/run	Trigger backup	admin:write	202, 400
GET	/administration/health	System health check	admin:read	200
POST	/administration/maintenance-mode	Enable maintenance mode	admin:write	200
GET	/administration/config	Get system config	admin:read	200

23.3 Audit Log Query Example

GET /administration/audit-logs?
 userId=usr_01HXY...&action=UPDATE&entity=Student&from=2026-07-01T00:00:00Z&to=2026-07-14T23:59:59Z&page=1&size=50

Response 200:

```
{
  "data": [
    {
      "id": "log_01HXY...",
      "userId": "usr_01HXY...",
      "userName": "Aarav Sharma",
      "action": "UPDATE",
      "entity": "Student",
      "entityId": "stu_01HXY...",
      "changes": {
        "firstName": {
          "from": "Aarav",
          "to": "Aaravkumar"
        }
      },
      "ipAddress": "192.168.1.100",
      "userAgent": "Mozilla/5.0...",
      "timestamp": "2026-07-14T14:30:00Z",
      "traceId": "abc-123"
    }
  ],
  "meta": {
    "page": 1,
```

```
    "size": 50,  
    "total": 145,  
    "totalPages": 3,  
    "hasNext": true,  
    "hasPrev": false  
  }  
}
```

24. Reports APIs

24.1 Reports Domain Overview

Reports domain हा read-only analytical views + report generation manage करतो. Reports async operations आहेत कारण large datasets process करणे synchronous impossible आहे. प्रत्येक report request 202 Accepted return करतो आणि background job मध्ये queued होतो. Job status polling endpoint via check केला जातो. Job complete झाल्यावर download URL available होतो जो 24 hours valid असतो. Standard reports pre-built आहेत; custom report builder business users साठी आहे जिथे filter + group + aggregate drag-drop interface द्वारे defined होतात.

Reports domain चे key aggregates: ReportJob (async job with status + result URL), ReportTemplate (reusable report definition), ScheduledReport (recurring report schedule with cron expression + recipients). Standard report categories: Attendance Reports (monthly, class-wise, student-wise), Finance Reports (collection summary, outstanding dues, fee structure analysis), Academic Reports (exam performance, progress card, term summary), Admissions Reports (funnel analysis, conversion rate, source attribution).

24.2 Reports Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/reports/student/attendance	Student attendance report	reports:read	202, 400
POST	/reports/finance/summary	Finance summary report	reports:read	202, 400
POST	/reports/admissions/funnel	Admissions funnel report	reports:read	202, 400
POST	/reports/academic/performance	Academic performance report	reports:read	202, 400
GET	/reports/jobs/{id}	Get report job status	reports:read	200, 404
GET	/reports/jobs/{id}/download	Download report	reports:read	200, 404, 410

Method	Path	Summary	Required Scope	Status Codes
POST	/reports/ custom	Custom report builder	reports:write	202, 400
GET	/reports/ templates	List report templates	reports:read	200
POST	/reports/ templates	Create report template	reports:write	201, 400
POST	/reports/ schedule	Schedule recurring report	reports:write	201, 400

24.3 Generate Report Example

POST /reports/student/attendance

Request:

```
{
  "studentIds": ["stu_01HXY..."],
  "fromDate": "2026-07-01",
  "toDate": "2026-07-31",
  "format": "PDF",
  "includeCharts": true
}
```

Response 202 (Accepted):

```
{
  "jobId": "job_rpt_01HXY...",
  "status": "pending",
  "estimatedCompletionTime": "2026-07-14T14:01:00Z"
}
```

Poll for completion:

GET /reports/jobs/job_rpt_01HXY...

Response 200 (when complete):

```
{
  "jobId": "job_rpt_01HXY...",
  "status": "completed",
  "progress": 100,
  "downloadUrl": "/reports/jobs/job_rpt_01HXY.../download",
  "expiresAt": "2026-07-15T14:00:00Z",
  "fileSize": 245678
  "mimeType": "application/pdf"
}
```

25. Platform APIs

25.1 Platform Domain Overview

Platform domain हा multi-tenant super-admin operations manage करतो — tenant management, subscriptions, billing, webhooks, API keys, platform-level metrics. हे सर्व endpoints फक्त Platform Admin role वाले users access करू शकतात. Tenant lifecycle: Trial → Active → Suspended → Terminated. Subscription lifecycle: Active → Past Due → Cancelled → Reactivated. Webhook registration द्वारे third-party systems PreOne events subscribe करू शकतात — हे OAuth2 client credentials flow सोबत authenticated असतात.

Platform domain चे key aggregates: Tenant (root customer organization with subscription + plan + features), Subscription (billing cycle with plan + limits + status), Webhook (event subscription with URL + signing secret + retries), ApiKey (per-tenant API key with scoped permissions). Platform metrics read-only aggregated views देतात — total tenants, active users, API calls per tenant, error rates. हे metrics real-time WebSocket channel द्वारे platform admin dashboard मध्ये push होतात.

25.2 Platform Endpoints

Method	Path	Summary	Required Scope	Status Codes
POST	/platform/tenants	Create tenant (super-admin)	platform:write	201, 400, 409
GET	/platform/tenants	List tenants	platform:read	200
PATCH	/platform/tenants/{id}	Update tenant	platform:write	200, 404
POST	/platform/subscriptions	Create subscription	platform:write	201, 400, 422
GET	/platform/subscriptions	List subscriptions	platform:read	200
POST	/platform/webhooks	Register webhook	platform:write	201, 400
GET	/platform/webhooks	List webhooks	platform:read	200

Method	Path	Summary	Required Scope	Status Codes
POST	/platform/webhooks/{id}/test	Test webhook delivery	platform:write	200, 404
GET	/platform/metrics	Platform metrics	platform:read	200
POST	/platform/api-keys	Generate API key	platform:write	201, 400

25.3 Register Webhook Example

POST /platform/webhooks

Request:

```
{
  "url": "https://api.partner.com/preone/webhooks",
  "events": [
    "student.admitted",
    "finance.payment.received"
  ],
  "description": "Sync to partner CRM"
}
```

Response 201:

```
{
  "id": "wh_01HXY...",
  "url": "https://api.partner.com/preone/webhooks",
  "events": ["student.admitted", "finance.payment.received"],
  "signingSecret": "whsec_abc123...",
  "status": "Active",
  "createdAt": "2026-07-14T15:00:00Z"
}
```

Test webhook delivery:

POST /platform/webhooks/wh_01HXY.../test

Response 200:

```
{
  "delivered": true,
  "responseStatus": 200,
  "responseTimeMs": 245,
  "attempt": 1
}
```

26. WebSocket Documentation

26.1 WebSocket Overview

PreOne real-time communication Socket.IO v4.x द्वारे handled होतो. WebSocket हा HTTP पेक्षा efficient bi-directional communication प्रदान करतो — server push + client push दोन्ही supported. PreOne मध्ये 5 documented WebSocket channels आहेत. प्रत्येक channel specific event types carry करतो आणि room-based subscription model वापरतो. Authentication JWT द्वारे connection handshake वर mandatory आहे — unauthenticated connections immediately disconnected.

WebSocket connection URL: `wss://api.preone.com/ws?token=<JWT>`. Connection handshake वर client JWT provide करतो, server validate करतो, आणि successful authentication वर connection established होतो. Rooms namespace-based isolation देतात — `branch:{bid}`, `user:{uid}`, `room:{roomId}`, `vehicle:{vehicleId}`. Client specific rooms मध्ये join होतो server-side event emit द्वारे. Message format JSON आहे with envelope: `{ event, payload, timestamp }`.

26.2 Documented Channels

Channel	Purpose	Room Pattern	Payload	Consumers
attendance.live	Real-time attendance updates per branch	branch:{bid}	AttendanceMarked event payload	Teacher dashboard live updates
notifications	User notifications push	user:{uid}	Notification payload + unread count	Web + mobile notification bell
chat	Two-way chat messages	room:{roomId}	Message payload + sender info	Parent-teacher chat
transport.live	Live transport vehicle tracking	vehicle:{vehicleId}	GPS coordinates + ETA + student status	Parent app + admin dashboard
dashboard.metrics	Live dashboard metric updates	branch:{bid};dashboard	Metric key + value + delta	Admin dashboard real-time cards

26.3 Connection Example

```
// Client-side (browser) connection
import { io } from 'socket.io-client';

const socket = io('https://api.preone.com/ws', {
  auth: {
    token: 'eyJhbGciOiJSUzI1NiIs... ',
    tenantId: 'ten_01HXY...',
    branchId: 'br_01HXY...'
  }
});

socket.on('connect', () => {
  console.log('Connected:', socket.id);
});

// Listen to attendance.live channel
socket.on('attendance.marked', (data) => {
  console.log('Attendance update:', data);
  // {
  //   studentIds: ['stu_01HXY...'],
  //   date: '2026-07-14',
  //   status: 'Present',
  //   markedBy: 'usr_01HXY...',
  //   markedAt: '2026-07-14T09:30:00Z'
  // }
});

// Listen to notifications channel
socket.on('notification.new', (data) => {
  console.log('New notification:', data);
});

socket.on('disconnect', (reason) => {
  console.log('Disconnected:', reason);
});

socket.on('error', (err) => {
  console.error('Socket error:', err);
});
```

27. Webhooks

27.1 Webhook Overview

Webhooks हे outbound HTTP callbacks आहेत जे PreOne च्या specific events occur झाल्यावर third-party systems ला notify करतात. WebSocket हे real-time push आहे पण ते PreOne च own clients साठी (web/mobile); Webhooks हे external systems साठी आहेत (partner CRMs, accounting software, parent apps). Webhook delivery at-least-once semantics द्वारे handled होतो — duplicate delivery possible आहे, clients idempotency handle करणे आवश्यक आहे via event id.

Webhook delivery retry strategy exponential backoff द्वारे: 1min, 5min, 30min, 2hr, 6hr, 24hr — total 6 attempts. हे सर्व attempts failed झाल्यास webhook auto-disabled होतो आणि tenant admin ला email notification जातो. Webhook payload HMAC-SHA256 द्वारे signed असतो — signature X-PreOne-Signature header मध्ये sent होतो. Clients signature verify करणे mandatory आहे to prevent forgery.

27.2 Webhook Events

Event Name	Event Type	Trigger	Payload	Consumers
StudentAdmitted	student.admitted	When admission approved + student record created	{ studentId, branchId, academicYearId, admittedAt }	CRM sync, welcome email trigger
PaymentReceived	finance.payment.received	When payment recorded (online or offline)	{ paymentId, invoiceId, studentId, amount, method, receivedAt }	Accounting sync, receipt email
AttendanceMarked	attendance.marked	When attendance marked (single or bulk)	{ studentIds[], date, status, markedBy, markedAt }	Parent SMS, daily summary

Event Name	Event Type	Trigger	Payload	Consumers
InvoiceGenerated	finance.invoice.generated	When invoice created (single or bulk)	{ invoiceId, studentId, amount, dueDate, generatedAt }	Parent notification, accounting sync
ReportCardPublished	academics.report.published	When report card published for student/class	{ studentId, classId, sectionId, termId, publishedAt }	Parent email, dashboard badge

27.3 Webhook Payload Example

```
# HTTP POST to registered webhook URL
POST https://api.partner.com/preone/webhooks HTTP/1.1
Host: api.partner.com
Content-Type: application/json
X-PreOne-Event: student.admitted
X-Preone-Event-Id: evt_01HXY...
X-Preone-Delivery: del_01HXY...
X-PreOne-Signature: sha256=abc123def456...
X-PreOne-Timestamp: 2026-07-14T12:00:00Z
User-Agent: PreOne-Webhook/1.0
```

```
{
  "event": "student.admitted",
  "eventId": "evt_01HXY...",
  "deliveryId": "del_01HXY...",
  "timestamp": "2026-07-14T12:00:00Z",
  "tenant": {
    "id": "ten_01HXY...",
    "name": "Sunshine Kids Group"
  },
  "data": {
    "studentId": "stu_01HXY...",
    "branchId": "br_01HXY...",
    "academicYearId": "ay_01HXY...",
    "admissionNo": "SKP-0001",
    "firstName": "Aarav",
    "lastName": "Sharma",
    "admittedAt": "2026-07-14T12:00:00Z"
  }
}
```

```
# Expected response from partner:
```

HTTP 2xx = success (no retry)
HTTP 4xx (except 429) = client error (no retry)
HTTP 5xx or 429 = server error (retry with backoff)

28. Examples

28.1 Per-Endpoint Example Matrix

प्रत्येक endpoint साठी standard set of examples provide केले जाते जे Swagger UI मध्ये render होतात आणि SDK consumers reference साठी वापरू शकतात. हे examples spec मध्ये inline नाहीत तर components/examples/ मध्ये reusable definitions म्हणून defined आहेत जे \$ref द्वारे referenced होतात. हे approach example consistency सुनिश्चित करतो — एक example अनेक endpoints reference करू शकतो.

Example Type	Description	Where Used
Request Example	Full request body + headers + query params	All POST/PATCH endpoints
Success Response (200)	Standard success body	All endpoints
Validation Error (400)	Field-level error array	All POST/PATCH endpoints
Permission Error (403)	Insufficient scope error	All secured endpoints
Not Found (404)	Resource not found error	All GET/PATCH/DELETE with {id}
Conflict (409)	Duplicate / version conflict	POST create + PATCH update
Business Rule Violation (422)	Domain invariant error	Workflow transition endpoints
Rate Limited (429)	Rate limit + Retry-After header	All endpoints
Internal Error (500)	Generic error + traceId	All endpoints
Pagination Example	page=2&size=20 + meta block	List endpoints
Filter Example	? filter=status:active,grade:LKG	List endpoints
Sorting Example	?sort=-createdAt,firstName	List endpoints
File Upload Example	multipart/form-data payload	Upload endpoints
Bulk Operation Example	Array of operations + bulk response	Bulk endpoints

28.2 Pagination Example

```
# Request with pagination
GET /students?page=2&size=20&sort=-createdAt,firstName HTTP/1.1
Host: api.preone.com
Authorization: Bearer eyJhbGc...

# Response 200:
{
  "data": [
    { "id": "stu_01HXY...", "firstName": "Aarav", ... },
    { "id": "stu_02HXY...", "firstName": "Diya", ... }
  ],
  "meta": {
    "page": 2,
    "size": 20,
    "total": 145,
    "totalPages": 8,
    "hasNext": true,
    "hasPrev": true
  }
}
```

28.3 Filter Example

```
# Multiple filters combined
GET /students?
filter=status:active,gender:Male,classId:cls_01HXY...&page=1&size=50
HTTP/1.1
Host: api.preone.com
Authorization: Bearer eyJhbGc...

# Filter syntax: field:value pairs joined by comma
# Equality filter: status:active
# IN filter: status:active,pending (interpreted as OR)
# Range filter: createdAt:2026-07-01..2026-07-31
# Null check: deletedAt:null
```

28.4 File Upload Example

```
# Multipart form data upload
POST /students/stu_01HXY.../photo HTTP/1.1
Host: api.preone.com
Authorization: Bearer eyJhbGc...
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="aarav.jpg"
Content-Type: image/jpeg
```

<binary data>
-----WebKitFormBoundary--

```
# Response 201:
{
  "fileId": "fil_01HXY...",
  "url": "https://cdn.preone.com/students/stu_01HXY.../photo.jpg",
  "size": 245678,
  "mimeType": "image/jpeg",
  "uploadedAt": "2026-07-14T16:00:00Z"
}
```

29. Tags

29.1 Tag Purpose

Tags हे Swagger UI मध्ये endpoints grouping करण्यासाठी वापरले जातात. PreOne मध्ये 14 business domains साठी 14 tags आहेत जे domain names शी match करतात. प्रत्येक tag चे description, color, आणि owner documented आहे. Tag-based grouping द्वारे Swagger UI मध्ये endpoints logical sections मध्ये organized दिसतात — user कोणतेही domain collapse किंवा expand करू शकतो.

Tag colors UI मध्ये visual distinction देतात — Identity blue, CRM violet, Students emerald, Finance green, इत्यादी. हे colors brand palette शी aligned आहेत आणि Design System document मध्ये documented आहेत. Tag descriptions Swagger UI मध्ये group header मध्ये display होतात ज्यामुळे user ला group purpose immediately समजतो.

29.2 Tag Catalog

Tag	Description	Color	Primary Users
Identity	User, role, permission management	Blue (#2563EB)	End-user + admin operations
CRM	Lead, campaign, follow-up management	Violet (#7C3AED)	Marketing + admissions team
Admissions	Admission application workflow	Amber (#D97706)	Admissions team + counselors
Students	Student record management	Emerald (#059669)	Branch admin + teachers
Academics	Classes, sections, subjects, timetable, exams	Cyan (#0891B2)	Academic coordinators
Attendance	Daily attendance + transport attendance	Rose (#E11D48)	Teachers + transport staff
Communication	SMS, WhatsApp, email, chat, notifications	Indigo (#4F46E5)	All staff + parents
Finance	Invoices, payments, receipts, refunds	Green (#16A34A)	Accountants + admin

Tag	Description	Color	Primary Users
Inventory	Items, stock, purchase orders, audit	Orange (#EA580C)	Inventory managers
HR	Employees, leaves, payroll, attendance	Pink (#DB2777)	HR team
Administration	Branches, academic year, holidays, audit	Slate (#475569)	Branch admin
Reports	Standard + custom reports, scheduling	Teal (#0D9488)	All staff
Settings	Tenant config, theme, modules	Gray (#374151)	Branch admin
Platform	Multi-tenant, subscriptions, webhooks	Crimson (#BE123C)	Platform admin

29.3 Tag Definition Example

tags:

- name: Identity
 - description: |
 - User, role, permission, and session management.
 - Foundation for all authenticated operations.
 - x-preone-color: '#2563EB'
 - x-preone-owner: identity-team@preone.com
 - x-preone-sla: 99.9%
- name: Students
 - description: |
 - Student records management — profile, guardians, address, academic history, fee history.
 - x-preone-color: '#059669'
 - x-preone-owner: student-team@preone.com
 - x-preone-sla: 99.95%

Usage in path:

paths:

/students:

get:

tags: [Students]

summary: List students

...

post:

tags: [Students]

summary: Create student

...

30. Versioning

30.1 Versioning Strategy

PreOne API URI versioning द्वारे versioned आहे — version URL path मध्ये (/api/v1, /api/v2). हा approach header-based versioning पेक्षा explicit आहे आणि SDKs साठी simpler आहे. URI versioning चा drawback म्हणजे breaking changes नवीन URL सोबत येतात जे old URLs सोबत parallel run होतात — हे operational cost आहे पण client compatibility साठी necessary आहे. Multiple major versions 12 months parallel supported असतात.

Spec version (info.version field) SemVer convention follow करतो — MAJOR.MINOR.PATCH. MAJOR bump द्वारे नवीन URL version (/api/v2) आणि 12-month deprecation cycle triggered होतो. MINOR bump द्वारे backward-compatible additions होतात जे same URL (/api/v1) मध्ये ship होतात. PATCH bump द्वारे bug fixes आणि doc updates होतात. Status header X-API-Version प्रत्येक response मध्ये sent होतो ज्यामुळे client आपला version pin किंवा upgrade करू शकतो.

30.2 Versioning Rules

Rule	Pattern	Behavior	Example
URI Versioning	/api/v1, /api/v2	MAJOR version in URL path	Breaking changes require new MAJOR
Semantic Versioning	MAJOR.MINOR.PATCH (1.0.0)	info.version field in spec	Follows semver.org rules
MAJOR Bump	Breaking change — schema/enum/route	New URL version + 12-month deprecation	Existing v1 frozen; v2 in parallel
MINOR Bump	New endpoint, new field (optional)	Same URL version	Backward compatible
PATCH Bump	Bug fix, doc clarification	Same URL version	No behavior change
Field Addition	Optional new field in response	MINOR bump	Old clients ignore unknown fields
Field Removal	Field marked deprecated then removed	MINOR → MAJOR (after 12 months)	Deprecation header returned

Rule	Pattern	Behavior	Example
Enum Addition	New enum value added	MINOR bump	Clients must handle unknown enum values
Enum Removal	Value marked deprecated then removed	MINOR → MAJOR (after 12 months)	Clients must validate enum
Status Header	X-API-Version: 1.0.0	Returned on every response	Client can pin or upgrade

30.3 Breaking Change Detection

CI pipeline मध्ये oasdiff tool द्वारा breaking change detection run होतो. हे tool PR मधील spec changes compare करून main branch शी breaking changes identify करतो. Breaking changes include: required field addition, field removal, type change, enum value removal, endpoint removal, URL change. Breaking changes detected झाल्यास PR block होतो आणि API Owner approval required होतो.

```
# CI pipeline breaking change detection
- name: Check for breaking changes
  run: |
    npx oasdiff breaking main.yaml pr.yaml \
      --format html \
      --output breaking-changes-report.html

    if [ -s breaking-changes-report.html ]; then
      echo "Breaking changes detected!"
      cat breaking-changes-report.html
      exit 1
    fi
```

31. Deprecation Policy

31.1 Deprecation Lifecycle

Endpoints आणि fields deprecate करण्याचा clear 12-month lifecycle documented आहे. हे lifecycle 6 stages मध्ये divided आहे जे gradually client migration enforce करतात. Stage 1 announcement सोबत सुरू होतो आणि Stage 6 (removal) सोबत complete होतो. या दरम्यान clients ला multiple warnings + time + tools provided होतात migration साठी. हे policy RFC 8594 (Sunset HTTP Header) शी aligned आहे.

Deprecation decision API Owner च्या approval नंतरच announce होतो. Major deprecations customer advisory board सोबत discussed होतात ज्यामुळे feedback gathered होतो. Per-tenant deprecation usage tracked राहतो — कोणता tenant कोणता deprecated endpoint वापरत आहे हे dashboard मध्ये visible आहे. Stage 4 (email notification) मध्ये प्रत्येक affected tenant ला personalized email जातो ज्यामध्ये त्यांच्या usage details आणि migration guide असते.

31.2 Deprecation Stages

Stage	Action	Timeline	Effect
Stage 1 — Announcement	Deprecation announced in changelog + spec x-deprecated: true	Day 0	Swagger UI shows strikethrough
Stage 2 — Warning Header	Deprecation + Sunset headers returned	Day 0 — 90	Clients see warnings in console
Stage 3 — Rate Limit Reduction	Deprecated endpoints rate-limited more aggressively	Day 90 — 180	Encourages migration
Stage 4 — Email Notification	Per-tenant email listing deprecated endpoints in use	Day 180 — 270	Active outreach
Stage 5 — Sunset Header	Sunset header with specific date	Day 270 — 365	Final warning
Stage 6 — Removal	Endpoint returns 410 Gone	Day 365+	Removed permanently

31.3 Deprecation Spec Example

```
# OpenAPI deprecation syntax
paths:
  /students/{id}/photo:
    get:
      deprecated: true
      summary: Get student photo (deprecated)
      description: |
        **DEPRECATED**: Use GET /students/{id}/assets/photo instead.
        This endpoint will be removed on 2027-07-14.
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Photo retrieved
          headers:
            Deprecation:
              schema:
                type: string
                description: 'true'
            Sunset:
              schema:
                type: string
                description: 'Sunset date: 2027-07-14'
            Link:
              schema:
                type: string
                description: '</students/{id}/assets/photo>; rel="successor-version"'
```

32. SDK Generation

32.1 SDK Targets

PreOne specification वरून 10 programming languages साठी SDKs auto-generate होतात. SDK generation CI pipeline मध्ये spec merge वर triggered होतो आणि package managers (npm, pub.dev, Maven Central, NuGet, PyPI, RubyGems, Packagist, etc.) वर published होतात. SDKs type-safe आहेत — TypeScript SDK full type definitions provide करतो, Java/C# SDKs POJOs/PONOs generate करतात. SDK versioning spec version शी synced आहे — spec v1.2.0 → SDK v1.2.0.

Primary SDK (most used) हा TypeScript SDK आहे जो PreOne च्या own frontend + mobile apps consume करतात. Secondary SDKs partner integrations साठी आहेत — partner system language नुसार appropriate SDK वापरतात. Mobile-first partners साठी Flutter/Dart SDK आहे, enterprise customers साठी Java/C# SDKs आहेत, data science use cases साठी Python SDK आहे. SDK documentation auto-generated + examples spec मधून sourced होतात.

32.2 SDK Targets Catalog

SDK Name	Package	Use Case	Generator	Distribution
TypeScript SDK	@preone/sdk-ts	Frontend + Node.js	Auto-generated via openapi-typescript-codegen	npm install
JavaScript SDK	@preone/sdk-js	Browser-only	Bundled TS, transpiled	CDN available
Python SDK	preone-python	Backend integrations + data science	Generated via openapi-python-client	pip install preone
Java SDK	io.preone:sdk-java	Enterprise integrations	Generated via openapi-generator	Maven Central
C# SDK	PreOne.Sdk	.NET integrations	Generated via openapi-generator (csharp)	NuGet

SDK Name	Package	Use Case	Generator	Distribution
Dart/Flutter SDK	preone_dart	Mobile app (Flutter)	Generated via openapi-generator (dart-dio)	pub.dev
Go SDK	github.com/preone/sdk-go	Microservices + CLI tools	Generated via oapi-codegen	go get
PHP SDK	preone/sdk-php	Legacy CMS integrations	Generated via openapi-generator (php)	Packagist
Ruby SDK	preone-sdk	Rails integrations	Generated via openapi-generator (ruby)	RubyGems
Swift SDK	PreOneSDK	iOS native app (future)	Generated via openapi-generator (swift5)	Swift Package Manager

32.3 TypeScript SDK Example

```
// npm install @preone/sdk-ts
import { PreOneClient } from '@preone/sdk-ts';

const client = new PreOneClient({
  environment: 'production', // 'production' | 'staging' | 'qa' | 'local'
  accessToken: 'eyJhbGciOiJSUzI1NiIs... ',
  tenantId: 'ten_01HXY...',
  branchId: 'br_01HXY...'
});

// List students with pagination
const response = await client.students.list({
  page: 1,
  size: 20,
  sort: '-createdAt',
  filter: 'status:active'
});

console.log(response.data); // Student[]
console.log(response.meta.total); // 145

// Create student
const newStudent = await client.students.create({
```

```
    firstName: 'Aarav',  
    lastName: 'Sharma',  
    dob: '2020-05-15',  
    gender: 'Male'  
  });
```

```
// Type-safe — IntelliSense + compile-time checks  
// All types auto-generated from OpenAPI spec
```

33. Validation Rules

33.1 Validation Standards

Swagger schema मध्ये validation rules declaratively defined आहेत. हे rules frontend (Zod), backend (class-validator), आणि SDKs सर्व consumers द्वारे enforced होतात. Validation layered approach follow करतो — frontend first validation करतो (instant feedback), backend re-validates (security), आणि database constraints final safety net आहेत. OpenAPI spec ही single source of truth आहे जिथून सर्व layers चे validation rules derived होतात.

Validation rules 14 categories मध्ये organized आहेत: required fields, string length, regex patterns, enum values, date formats, UUID format, email format, URI format, decimal precision, numeric range, array constraints, object constraints, nullable fields, आणि custom validators. Custom validators domain-specific rules साठी आहेत — उदा. mobileNumber (10-digit Indian mobile), aadhaarNumber (12-digit + Verhoeff algorithm), panNumber (10-char alphanumeric + checksum).

33.2 Validation Rules Catalog

Rule Type	Spec Syntax	Description	Backend Enforcement
Required Fields	required: [firstName, dob, gender]	Spec enforces presence at request level	Backend re-validates
String Length	minLength / maxLength	E.g., firstName: 1-100 chars	Backend rejects with VALIDATION_ERROR
Regex Patterns	pattern: '^[A-Z]{3}-\d{4}\$'	PCRE-compatible regex	Used for admission numbers, codes
Enum Values	enum: [Male, Female, Other]	Closed enum — additional values rejected	Backend re-validates against domain enum
Date Formats	format: date (YYYY-MM-DD)	ISO 8601 strict	date-time for timestamps
UUID Format	format: uuid	RFC 4122 v4	All entity identifiers
Email Format	format: email	RFC 5322 simplified	Backend uses validator.js

Rule Type	Spec Syntax	Description	Backend Enforcement
URI Format	format: uri	Must be valid absolute URL	Used for webhooks, file URLs
Decimal Precision	multipleOf: 0.01	Currency precision	Backend uses Decimal.js
Numeric Range	minimum / maximum	Inclusive bounds	exclusiveMinimum for strict bounds
Array Constraints	minItems / maxItems / uniqueItems	E.g., bulk max 100 items	Backend enforces hard limit
Object Constraints	minProperties / maxProperties	E.g., filter object max 10 keys	Prevents query explosion
Nullable Fields	type: [string, null]	OpenAPI 3.1 nullable syntax	Backend treats null as absent
Custom Validators	x-preone-validator: mobileNumber	Extension for domain validators	Backend uses custom Zod refinements

33.3 Validation Example

```

components:
  schemas:
    StudentCreateRequest:
      type: object
      required: [firstName, mobile, dob, gender]
      properties:
        firstName:
          type: string
          minLength: 1
          maxLength: 100
          pattern: '^[a-zA-Z\s]+$'
          description: Alphabetic + spaces only
        mobile:
          type: string
          pattern: '^[6-9]\d{9}$'
          description: 10-digit Indian mobile starting 6-9
          x-preone-validator: mobileNumber
        dob:
          type: string
          format: date
          x-preone-validator: notFutureDate
        gender:
          type: string
          enum: [Male, Female, Other]

```

```

email:
  type: [string, null]
  format: email
  description: Optional, must be valid email if provided
admissionNo:
  type: string
  pattern: '^[A-Z]{3}-\\d{4}$'
  description: 3 uppercase letters + dash + 4 digits (e.g., SKP-0001)
guardians:
  type: array
  minItems: 1
  maxItems: 4
  uniqueItems: true
  items:
    $ref: '#/components/schemas/GuardianRequest'
additionalProperties: false

```

```

# Generated TypeScript type (auto-generated):
interface StudentCreateRequest {
  firstName: string;      // 1-100 chars, alphabetic+spaces
  mobile: string;         // 10-digit Indian mobile
  dob: string;            // ISO date, not future
  gender: 'Male' | 'Female' | 'Other';
  email?: string | null;  // RFC 5322 email
  admissionNo?: string;   // SKP-0001 format
  guardians: GuardianRequest[]; // 1-4 unique items
}

```

34. Governance

34.1 Governance Overview

API governance हा process आहे ज्याद्वारे API spec changes controlled, reviewed, आणि published होतात. PreOne च्या governance model मध्ये 6 defined roles आहेत ज्यांच्या पास specific responsibilities आहेत. हे roles ensure करतात की spec changes high quality, secure, compliant, आणि developer-friendly आहेत. कोणताही change फक्त PR-based approval workflow द्वारे spec मध्ये merge होतो — direct commits strictly forbidden आहेत.

Governance workflow 10 steps मध्ये divided आहे जे spec change announcement पासून customer notification पर्यंत complete journey define करतात. प्रत्येक step automated जिथे possible आहे — linting, breaking change detection, SDK regeneration, Swagger UI deployment. Manual steps review sign-off, changelog edit, आणि customer communication आहेत. हे workflow API quality + reliability सुनिश्चित करतो.

34.2 Governance Roles

Role	Held By	Responsibilities	Approval Authority
API Owner	Product Manager	Owns endpoint lifecycle, deprecation, breaking changes	Approval required for MAJOR bumps
API Architect	Tech Lead	Owns spec structure, naming, common components	Approval required for new tags
Spec Maintainer	Senior Engineer	Day-to-day spec edits, PR reviews	Maintains openapi.yaml
Security Reviewer	Security Engineer	Reviews auth flows, scopes, sensitive endpoints	Required sign-off for new auth schemes
DX Reviewer	DX Engineer	Reviews SDK quality, examples, docs	Required sign-off for public endpoints
Compliance Reviewer	Compliance Officer	Reviews PII handling, audit logging	Required for endpoints touching student/finance data

34.3 Governance Workflow

Step	Action	Notes
1. Spec Change PR	Author submits PR against openapi.yaml	All changes via PR — no direct commits
2. Automated Lint	Redocly CLI + Spectral run in CI	Must pass — no warnings allowed
3. Breaking Change Detection	oasdiff runs against main branch	Breaking changes flag for API Owner review
4. Reviewer Assignment	Auto-assigned based on changed tags	1 reviewer minimum; 2 for breaking changes
5. Review Sign-off	Reviewers approve	All comments resolved
6. Merge to Main	Squash merge to main	Spec frozen on merge
7. SDK Regeneration	CI generates SDKs for all targets	SDKs published to package managers
8. Swagger UI Update	Swagger UI redeployed	Live at developers.preone.com
9. Changelog Update	Auto-generated changelog PR	Manual edit for narrative
10. Customer Notification	For MINOR/MAJOR bumps — email + dashboard banner	30-day advance for MAJOR bumps

34.4 Governance Tools

```
# CI pipeline (GitHub Actions)
name: OpenAPI Spec Validation
on:
  pull_request:
    paths: ['specs/**']

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Setup Node
        uses: actions/setup-node@v4
```

```

with:
  node-version: 20

- name: Install Redocly CLI
  run: npm install -g @redocly/cli@latest

- name: Lint OpenAPI spec
  run: redocly lint specs/openapi.yaml --config redocly.yaml

- name: Bundle spec
  run: redocly bundle specs/openapi.yaml --output bundled.yaml

- name: Detect breaking changes
  run: |
    git fetch origin main
    npx oasdiff breaking origin/main:specs/openapi.yaml specs/openapi.yaml \
      --format text

- name: Validate examples
  run: npx @stoplight/spectral-cli lint specs/openapi.yaml

- name: Generate SDKs (dry-run)
  run: |
    npx openapi-typescript-codegen --input bundled.yaml \
      --output /tmp/sdk-ts --client axios

- name: Post comment with summary
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '✅ OpenAPI spec validation passed'
      })

```

35. Generated Deliverables

35.1 Automation Pipeline

या specification वरून automate होणारे artifacts ही document चा primary value proposition आहे. Manual maintenance न करता spec एक बदलला की सर्व downstream artifacts regenerate होतात. हे automation CI/CD pipeline द्वारे handled होतो जो spec merge वर triggered होतो. प्रत्येक artifact ची quality gate defined आहे — SDKs चे tests run होतात, Postman collection चे contract tests run होतात, Swagger UI चे visual regression tests run होतात.

Artifact generation timing different आहे. Swagger UI + Postman Collection immediate deploy होतात (within minutes of merge). SDKs चे publish हे release tag वर triggered होतो (scheduled releases weekly). Mock server continuously updated होतो with latest spec. Contract tests every CI run मध्ये executed होतात. हे staged approach balance देतो — quick iteration साठी immediate artifacts, stability साठी release-gated SDKs.

35.2 Deliverables Catalog

Artifact	Purpose	Location	Update Trigger
Swagger UI	Interactive API explorer	Hosted at developers.preone.com/api-explorer	Auto-deployed from main
OpenAPI 3.1 YAML	Canonical spec file	openapi.yaml in monorepo	Source of truth
OpenAPI JSON	Machine-readable spec	openapi.json (generated)	For tools that prefer JSON
Postman Collection	Importable collection + environments	PreOne.postman_collection.json	Per-environment variables
Mock Server	Prism-based mock server	Hosted at mock.preone.com	For frontend + mobile dev
TypeScript SDK	Generated TS client	@preone/sdk-ts on npm	Auto-published on merge
Flutter/Dart SDK	Generated Dart client	preone_dart on pub.dev	For mobile app
C# SDK	Generated C# client	PreOne.Sdk on NuGet	For enterprise integrations

Artifact	Purpose	Location	Update Trigger
Java SDK	Generated Java client	io.preone:sdk-java on Maven	For enterprise integrations
API Test Collections	Contract + integration tests	Postman + Newman CI	Runs on every spec change
Contract Validation	Pact-style contract tests	Backend implements + spec validates	Bidirectional contract testing
Client Code Generation	Auto-generated clients for all languages	10 target languages	Per release tag

35.3 Quality Gates

प्रत्येक generated artifact चे quality gate आहे जे ensure करते की artifact production-ready आहे:

- Swagger UI — visual regression tests via Playwright
- Postman Collection — Newman contract tests against staging
- Mock Server — Prism schema validation against spec
- TypeScript SDK — generated types compile + sample app builds
- Dart SDK — flutter analyze + sample app builds
- Java SDK — mvn test + sample integration test
- C# SDK — dotnet test + sample integration test
- Contract Tests — backend implementation matches spec

36. Glossary

36.1 Terms

Term	Definition
OpenAPI	Specification for describing HTTP APIs (formerly Swagger Spec)
Swagger UI	Interactive UI for browsing OpenAPI specs
JWT	JSON Web Token — compact, self-contained auth token
RBAC	Role-Based Access Control
Scope	Granular permission identifier (e.g., student:read)
Aggregate	Cluster of domain objects treated as a unit (DDD)
Webhook	HTTP callback triggered by an event
WebSocket	Persistent bi-directional communication channel
Idempotency Key	Client-generated UUID for safe retry of write operations
Rate Limiting	Restricting request count per client per time window
Tenant	Customer organization in a multi-tenant system
Branch	Sub-tenant unit (a single preschool location)
Academic Year	Yearly academic period (e.g., 2025-2026)
Deprecation	Marking an endpoint as scheduled for removal
Sunset Header	HTTP header announcing future endpoint removal date
SDK	Software Development Kit — client library for consuming an API
Contract Test	Test verifying API conforms to its specification

Term	Definition
Mock Server	Server mimicking API behavior from spec, for development
Discriminator	OpenAPI field for choosing schema variant by type field
\$ref	Reference to a reusable component within or external to spec

37. Document Control

37.1 Document Information

Document Title	PreOne Enterprise OpenAPI Specification
Document Version	1.0
Status	API Specification Freeze
Specification	OpenAPI 3.1
Date	2026-07-14
Product	PreOne — Enterprise Preschool Operating System
Predecessor	Vision v1.0, Master PRD v1.0, DDD v1.0, ERD v3.0, API Contract Catalog v1.0
Successor	Swagger UI, Postman Collection, SDK Generation, Mock Server, Backend Validation, API Testing
Classification	Internal Engineering Reference
Prepared by	PreOne Architecture & Engineering Team

37.2 Change History

Version	Date	Author	Changes
1.0	2026-07-14	Architecture Team	Initial freeze — 34 sections covering 14 domains, ~530 APIs, WebSocket, Webhooks, SDK generation, governance
0.9	2026-07-10	Architecture Team	Pre-freeze draft — internal review feedback incorporated
0.5	2026-07-01	Architecture Team	Initial draft — sections 1-12 (standards, auth, components)
0.1	2026-06-15	Architecture Team	Skeleton — TOC + section headers + placeholder content

37.3 Approval Matrix

Role	Name	Responsibility	Status
API Owner	—	Endpoint lifecycle, deprecation decisions	Pending
API Architect	—	Spec structure, naming, components	Pending
Security Reviewer	—	Auth flows, scopes, sensitive endpoints	Pending
DX Reviewer	—	SDK quality, examples, documentation	Pending
Compliance Reviewer	—	PII handling, audit logging	Pending

37.4 Next Steps

- Architecture review board sign-off (this week)
- Security review for OAuth2 + SSO roadmap (next sprint)
- Engineering team walkthrough + Q&A session
- Partner integration team enablement + SDK onboarding
- Customer advisory board notification of v1.0 freeze
- Backend implementation kickoff — controllers + DTOs from spec
- Frontend + mobile SDK consumption patterns documentation
- Contract test suite development against spec
- Postman Collection + Newman CI integration
- Swagger UI deployment to developers.preone.com